PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

Early Detection of Malicious URLs Using Machine Learning Algorithms

A PROJECT REPORT

Submitted by

BHAGYA LAKSHMI- 20221IST0058

SANJANA C RAO- 20221IST0072 YASHASWINI D- 20221IST0083

Under the guidance of,

Dr.Debasmita Mishra

BACHELOR OF TECHNOLOGY

IN

INFORMATION SCIENCE AND TECHNOLOGY

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report "Early Detection of Malicious URLs Using Machine Learning Algorithms" is a bonafide work of "Bhagyalakshmi (20221IST0058), Sanjana C Rao(20221IST0054), YASHASWINI D (20221IST0083)", who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in INFORMATION SCIENCE AND TECHNOLOGY during 2025-26.

Dr.Debasmita Mishra
Project Guide
PSCS Presidency University

Mrs. Benitha Christinal J
Program Project Coordinator
PSCS
Presidency University

Dr. Sampath A K
School Project Coordinators
PSCS
Presidency University

Dr. Pallavi R
Head of the Department
PSCS
Presidency University

Dr. Duraipandian N
Associate Dean PSCS
Presidency University

Name and Signature of the Examiners

1)

2)

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We the students of final year B.Tech in INFORMATION SCIENCE AND TECHNOLOGY at Presidency University, Bengaluru, named Bhagyalakshmi, Sanjana C Rao, Yashaswini D, hereby declare that the project work titled "Early Detection of Malicious URLs Using Machine Learning Algorithms" has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in INFORMATION SCIENCE AND TECHNOLOGY during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Bhagyalakshmi USN: 20221IST0058

Sanjana C Rao USN: 20221IST0072

Yashaswini D USN: 20221IST0083

PLACE: BENGALURU DATE:

ACKNOWLEDGEMENT

For completing this project work, we have received the support and the guidance from many people whom we would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

We would like to sincerely thank my internal guide **Dr.Debasmita Mishra, Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University, for her moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

We am also thankful to **Dr. Pallavi R, Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for her mentorship and encouragement.

We express our cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, Presidency School of computer Science and Engineering and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Dr. Sampath A K, PSCS** Project Coordinators**, Mrs. Benitha Christinal, Program Project Coordinator**, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

BHAGYALAKSHMI
SANJANA C RAO
YASHASWINI D

ABSTRACT

Malicious URLs are one of the most common entry points for cyberattacks, including phishing, malware distribution, and data theft. With the rapid growth of internet usage, traditional detection methods such as blacklist-based systems are no longer sufficient, as they fail to identify newly generated or obfuscated URLs in real time. This creates a critical need for intelligent and efficient systems that can detect malicious links accurately and quickly.

This project presents a machine learning-based approach for real-time malicious URL detection. The system analyzes URLs by extracting multiple features related to their structure, domain information, and behavior. A total of 54 features are considered, including lexical, host-based, content-based, and network-based attributes. These features are preprocessed and used to train an ensemble model that combines techniques such as Random Forest, Support Vector Machine (SVM), and Long Short-Term Memory (LSTM) networks to improve prediction accuracy and robustness.

The dataset used in this project consists of a large number of labeled URLs, including both safe and malicious instances, ensuring that the model learns diverse patterns. The data is split into training, validation, and testing sets to evaluate performance effectively. The system is designed to process inputs in real time, making it suitable for applications such as browser extensions and web security tools.

The results demonstrate that the proposed model achieves high accuracy in classifying URLs, with low false positive rates and fast response times. The system is capable of identifying suspicious patterns and providing clear output to users, including risk indications for potentially harmful links. Overall, the project provides an effective and scalable solution for enhancing online security by detecting malicious URLs efficiently and reliably.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ACKNOWLEDGEMENT	**iv**
	ABSTRACT	**v**
	TABLE OF CONTENTS	**vi**
	LIST OF FIGURES	**viii**
	LIST OF TABLES	**ix**
1.	**INTRODUCTION**	1
	1.1 BACKGROUND AND MOTIVATION	1
	1.2SCOPE AND LIMITATIONS	3
	1.3 PROBLEM STATEMENT	4
	1.4 MOTIVATION	5
	1.5METHODOLOGY OVERVIEW	6
	1.6ORGANIZATION OF THE THESIS	8
	1.6.1Logical Flow of the Research Work	8
	1.6.2Contribution Mapping Across Chapters	9
	1.6.3Chapter Overview Summary	10
2.	**LITERATURE REVIEW**	12
	2.1 INTRODUCTION TO RELATED WORK	12
	2.2 SURVEY OF EXISTING SYSTEMS	13
	2.3 RESEARCH GAPS IDENTIFIED	15
	2.4 OBJECTIVES OF THE STUDY	16
	2.5 SUMMARY	16

3.	**RESEARCH METHODOLOGY**	18
	3.1 RESEARCH METHODOLOGY OUTLINE	18
	3.2 TOOLS, TECHNOLOGIES, PLATFORMS USED	18
	3.3 PROPOSED ARCHITECTURE	19
	3.4 DATA COLLECTION & PREPROCESSING	21
	3.4.1 Data source and composition	21
	3.4.2 Feature selection and structuring	21
	3.4.3 Preprocessing workflow	21
	3.4.4 Sequence construction for temporal learning	22
	3.4.5 Data splitting and validation strategy	22
	3.5DATA SET DESCRIPTION	22
	3.5.1 Data Acquisition and Understanding	22
	3.5.2 Feature Engineering and Sequence Construction	23
	3.5.3 Synthetic Data Generation Usi	24
	3.6 WORKFLOW DIAGRAM	24
	3.7 SUMMARY	26
4.	**IMPLEMENTATION**	29
	4.1 MODULE-WISE IMPLEMENTATION DETAILS	29
	4.1.1 Data Preparation and Encoding Module	29
	4.1.2 Sequence Construction and Windowing Module	30
	4.1.3 Model Architecture and Model Training	32
	4.2 ALGORITHMS	33
	4.3 SCREENSHOTS	35
	4.4 INTEGRATION OF COMPONENTS	41

	4.5 SUMMARY	43
5.	**RESULTS AND DISCUSSION**	45
	5.1 EXPERIMENTAL SETUP ANDENVIORNMENT	45
	5.2 PERFORMANCE METRICS USED	46
	5.3 COMPARATIVE RESULTS	48
	5.4 ANALYSIS AND INTERPRETATION OFRESULTS	49
	5.5 DISCUSSION ON OUTCOMES	50
	5.6 SUMMARY	51
6.	**CONCLUSION AND FUTURE WORK**	52
	6.1 SUMMARY OF CONTRIBUTIONS	52
	6.2 KEY FINDINGS	53
	6.3 LIMITATIONS OF THE STUDY	54
	6.4 SUGGESTIONS FOR FUTURE RESEARCH	55
	6.5 SUMMARY	57
	REFERENCES	59
	BASE PAPER	59
	APPENDIX- A	60
	APPENDIX- B	60
	APPENDIX- C	62
	APPENDIX- D	62

LIST OF FIGURES

Sl. No.	Figure Name	Caption	Page No.
1	Figure 3.1	System archietecture	20
2	Figure 3.2	Data WorkFlow	26
3	Figure 4.1	Modules Overview	31
4	Figure 4.2	Main Dashboard and Query Interface of the System	36
5	Figure 4.3	Safe URL Detection Output	37
6	Figure 4.4	Classification Result for Benign URL	39
7	Figure 4.5	System Output for Legitimate URL Analysis	41
8	Figure A.1	Similarity Report	61
9	Figure C.1	Research Paper	62
10	Figure D.1	Project Github Repository	62

LIST OF TABLES

Sl. No.	Table Name	Table Caption	Page No.
1	Table 2.1	Summary of Literature Reviewed	14
2	Table 3.1	Tools,Technologies,and Platforms used	19
3	Table 3.5	Features table	24
4	Table 4.1	System Modules Overview	33
5	Table 5.1	Comparision Table	49

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
URL	Uniform Resource Locator
SVM	Support Vector Machine
ML	Machine Learning
IP	Internet Protocol
API	Application Programming Interface
CSV	Comma Separated Values
UI	User Interface
AUC	Area Under Curve
ROC	Receiver Operating Characteristic
TPR	True Positive Rate
FPR	False Positive Rate
API	Application Programming Interface
WHOIS	Internet domain registration lookup protocol
HTML	HyperText Markup Language
CSS	Cascading Style Sheets

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

In recent years, the rapid growth of the internet has fundamentally transformed how individuals, businesses, and governments operate. From online banking and e-commerce to cloud computing and digital healthcare, the internet has become an essential infrastructure supporting modern society. However, this digital transformation has also led to a significant increase in cybersecurity threats, particularly those exploiting web-based platforms. Among these threats, malicious URLs have emerged as one of the most common and dangerous attack vectors used by cybercriminals.

A malicious URL is a web link designed to deceive users or compromise their systems by delivering harmful content such as phishing pages, malware, ransomware, or spyware. These URLs often appear legitimate and are distributed through emails, social media, advertisements, and messaging platforms. As internet usage continues to expand globally, attackers are becoming more sophisticated in crafting URLs that closely resemble trusted domains, making it increasingly difficult for users to distinguish between safe and malicious links.

Traditional approaches to detecting malicious URLs, such as blacklist-based systems, have been widely adopted by security organizations and web browsers. These systems maintain a database of known malicious URLs and block access when a match is found. While effective for previously identified threats, blacklist-based methods suffer from a critical limitation — they are reactive rather than proactive. Newly created malicious URLs, often referred to as zero-day threats, can remain undetected for several hours or even days before being added to the blacklist. During this time, a large number of users may fall victim to cyberattacks.

In addition to blacklists, rule-based detection systems were introduced to improve security by analyzing patterns in URL structures. These systems rely on manually crafted rules to identify suspicious characteristics such as unusual domain names, excessive subdomains, or the presence of special characters. Although rule-based systems provide faster detection compared to blacklists, they are highly dependent on expert knowledge and struggle to adapt to the

continuously evolving tactics used by attackers. Maintaining and updating these rules also becomes increasingly complex and inefficient at scale.

To overcome these challenges, the field of cybersecurity has increasingly turned toward Machine Learning (ML) as a more adaptive and intelligent solution. Machine learning algorithms can analyze large volumes of URL data, identify hidden patterns, and automatically learn the distinguishing characteristics of malicious URLs. Unlike traditional methods, ML-based systems can generalize from past data and detect previously unseen threats, making them particularly effective for early-stage detection.

Recent advancements in machine learning, including ensemble learning and deep learning techniques, have further enhanced the capabilities of URL classification systems. These approaches allow the integration of multiple types of features, such as lexical features (URL structure), host-based features (domain registration and DNS information), content-based features (HTML and JavaScript analysis), and network-based features (traffic behavior). By combining these diverse feature sets, ML models can achieve higher accuracy and robustness in identifying malicious URLs.

Despite these advancements, several challenges remain. Cyber attackers continuously evolve their strategies using techniques such as URL obfuscation, domain generation algorithms (DGAs), and fast-flux networks to evade detection. Moreover, achieving high detection accuracy while maintaining low latency is crucial for real-time applications such as browser extensions and network security systems. There is also a need to minimize false positives to avoid blocking legitimate websites, which can negatively impact user experience.

The motivation for this project arises from the urgent need to develop an efficient, accurate, and real-time system for detecting malicious URLs. By leveraging machine learning algorithms, this project aims to bridge the gap between traditional detection methods and modern cybersecurity requirements. The focus is on building a system that not only detects known threats but also identifies new and evolving malicious URLs at an early stage.

Furthermore, with the increasing reliance on digital platforms in everyday life, ensuring user safety has become a critical concern. Individuals often lack the technical expertise to recognize malicious links, making them vulnerable to cyberattacks. Therefore, developing an automated

detection system that can operate seamlessly in the background and provide realtime protection is highly valuable**.**

1.2 SCOPE AND LIMITATIONS

The scope of this project is centered around the design and development of an intelligent system capable of identifying malicious URLs at an early stage using machine learning techniques. The system focuses on analysing URLs before a user interacts with them, thereby preventing potential cyber threats such as phishing, malware downloads, and data theft. The primary objective within this scope is to create a model that can classify URLs as either benign or malicious based on multiple extracted features.

The project mainly operates within the domain of **web security and machine learning**, combining concepts from data analysis, cybersecurity, and artificial intelligence. It involves collecting a labelled dataset of URLs, performing feature extraction, training classification models, and evaluating their performance using standard metrics such as accuracy, precision, recall, and latency. The system is designed to function in near real-time, making it suitable for applications such as browser extensions, email filtering systems, and network security tools.

A key part of the project scope includes the use of different categories of features. Lexical features derived directly from the URL string are used for fast and efficient detection. Hostbased features such as domain age and DNS records provide additional context about the legitimacy of the source. Content-based features, although more computationally intensive, help identify malicious intent embedded within web pages. By combining these features, the system aims to achieve a balance between accuracy and performance.

Another important aspect of the scope is the implementation of multiple machine learning algorithms, such as Random Forest, Support Vector Machine (SVM), and possibly deep learning models like LSTM. These models are compared and combined to improve detection performance. The project also explores the feasibility of deploying the system as a lightweight prototype, demonstrating its practical usability.

However, despite covering a wide range of functionalities, the project has certain limitations. One of the primary limitations is the dependency on the quality and size of the dataset. If the dataset does not adequately represent real-world malicious URL patterns, the model's performance may be affected. Additionally, datasets may become outdated over time due to the constantly evolving nature of cyber threats.

Another limitation lies in the computational cost of certain features. Content-based analysis requires accessing and processing web page data, which increases latency and may not be suitable for real-time applications in all scenarios. Therefore, the system may rely more heavily on faster features like lexical and host-based attributes for initial detection.

The system may also face challenges in detecting highly sophisticated attacks that use advanced obfuscation techniques or mimic legitimate URLs very closely. Adversarial attacks, where attackers intentionally design URLs to bypass detection models, remain a significant challenge in machine learning-based cybersecurity systems.

Furthermore, the current implementation focuses on URL classification and does not extend to full threat mitigation or incident response. It identifies potential threats but does not automatically remove or neutralize them. Integration with larger security systems would be required for complete protection.

In summary, while the project covers the design of an effective and practical malicious URL detection system, it is constrained by dataset limitations, computational trade-offs, and evolving attack strategies. These limitations highlight opportunities for future improvements and extensions.

1.3 PROBLEM STATEMENT

With the exponential growth of internet usage, cyber threats exploiting web technologies have increased significantly. Among these threats, malicious URLs serve as a primary entry point for attacks such as phishing, malware distribution, and data breaches. These URLs are often designed to appear legitimate, making it difficult for users to identify and avoid them.

Traditional detection mechanisms, such as blacklist-based systems and rule-based approaches, are insufficient to address modern cybersecurity challenges. Blacklists fail to detect newly created malicious URLs in real time, while rule-based systems struggle to adapt to evolving attack patterns. As a result, there is a critical need for a more dynamic and intelligent approach to detect malicious URLs effectively.

The main problem addressed in this project is the **lack of an efficient, accurate, and realtime system capable of detecting both known and unknown malicious URLs**. Existing systems either compromise on speed or fail to generalize well to new threats. Additionally, high false positive rates can lead to blocking legitimate websites, affecting user experience.

1.4 MOTIVATION

The motivation for this project arises from the rapidly increasing dependence on digital platforms in everyday life and the corresponding rise in cyber threats. Today, users rely heavily on the internet for activities such as online banking, shopping, education, communication, and cloud-based services. While this digital convenience has improved efficiency and accessibility, it has also exposed users to a wide range of security risks, particularly those originating from malicious URLs.

One of the primary concerns is that most users lack the technical knowledge required to identify suspicious or harmful links. Cyber attackers exploit this gap by designing URLs that closely resemble legitimate websites, often using slight variations in spelling, domain extensions, or structure. As a result, even cautious users may unknowingly click on malicious links, leading to data theft, financial loss, or system compromise. This highlights the need for an automated detection mechanism that operates independently of user awareness.

Another key motivating factor is the limitation of traditional security systems. Blacklist-based approaches, although widely used, are inherently reactive. They depend on prior identification and reporting of malicious URLs, which creates a delay in detection. During this delay, newly created malicious URLs can cause significant damage. Similarly, rule-based systems are rigid and struggle to keep up with the constantly evolving techniques used by attackers. These limitations make it clear that more adaptive and intelligent solutions are required.

Machine learning provides a promising direction for addressing these challenges. Unlike traditional approaches, machine learning models can learn patterns from large datasets and generalize to detect previously unseen threats. This ability to adapt and improve over time makes ML-based systems highly suitable for cybersecurity applications. The motivation behind this project is to leverage these capabilities to build a system that can detect malicious URLs more effectively and efficiently.

In addition, there is a growing demand for real-time security solutions. In many scenarios, such as web browsing or email filtering, decisions need to be made within milliseconds. A delay in detection can result in users accessing harmful content. Therefore, the system developed in this project aims to achieve not only high accuracy but also low latency, ensuring that it can be used in real-world applications without affecting user experience.

Another important motivation is the practical relevance and scalability of the solution. A machine learning-based URL detection system can be easily integrated into various platforms such as web browsers, enterprise security systems, and mobile applications. Once deployed, the system can continuously learn from new data and improve its performance over time, making it a sustainable and future-ready solution.

From an academic perspective, this project also provides an opportunity to apply theoretical concepts of machine learning and data analysis to a real-world problem. It bridges the gap between academic learning and practical implementation, allowing for a deeper understanding of both domains.

1.5 METHODOLOGY OVERVIEW

The methodology adopted for this project follows a structured and systematic approach to ensure accurate and efficient detection of malicious URLs. The overall process involves multiple stages, including data collection, preprocessing, feature extraction, model training, evaluation, and deployment.

The first stage involves collecting a dataset of URLs from reliable sources. The dataset includes both benign and malicious URLs, ensuring that the model learns to differentiate between safe

and harmful links. Data quality plays a crucial role in this stage, as the performance of the model heavily depends on the diversity and accuracy of the dataset.

Once the data is collected, preprocessing is performed to clean and organize the dataset. This includes removing duplicates, handling missing values, and normalizing the data. Preprocessing ensures that the dataset is suitable for further analysis and model training.

The next stage is feature extraction, which is a critical component of the methodology. In this step, meaningful attributes are derived from the URLs. These features are categorized into different types. Lexical features are extracted directly from the URL string and include characteristics such as length, number of special characters, and presence of suspicious patterns. Host-based features provide information about the domain, such as domain age, DNS records, and IP address details. In some cases, content-based features may also be considered by analysing the webpage content associated with the URL.

After feature extraction, the dataset is divided into training, validation, and testing sets. This division helps in evaluating the model's performance on unseen data and prevents overfitting. Machine learning algorithms such as Random Forest and Support Vector Machine (SVM) are then applied to the training data. These algorithms are chosen for their ability to handle classification problems effectively and provide high accuracy.

In addition to traditional machine learning models, deep learning approaches such as Long Short-Term Memory (LSTM) networks may also be explored. These models are particularly useful for capturing sequential patterns in URLs, which can further improve detection performance.

Once the models are trained, they are evaluated using standard performance metrics such as accuracy, precision, recall, and F1-score. These metrics provide a comprehensive understanding of the model's effectiveness, including its ability to correctly identify malicious URLs and minimize false positives.

The final stage involves deploying the model as a prototype system. This may include integrating the model into a simple application or browser extension that can analyse URLs in real time. The deployment stage demonstrates the practical usability of the system and validates its performance in real-world scenarios.

Overall, the methodology ensures a balance between accuracy, efficiency, and scalability, making the system suitable for practical cybersecurity applications.

1.6 THESIS ORGANISATION

The project report is systematically organized to present the research work in a clear, logical, and structured manner. Each chapter is carefully designed to address a specific component of the project, ensuring a smooth flow of information from the identification of the problem to the final conclusions and future directions. This structured organization helps in understanding the progression of the work and the relationship between different stages of the project.

The report begins with an introduction that establishes the context of the study, followed by a detailed analysis of existing research, the proposed methodology, system design, implementation, evaluation, and finally the broader implications of the project. This logical arrangement ensures that the reader can follow the development of the project step by step.

1.6.1 Logical Flow of the Research Work

The logical flow of the research work in this project follows a systematic problem-solving approach. It begins with identifying the growing issue of malicious URLs and understanding their impact on cybersecurity. This is followed by an exploration of existing detection techniques, including blacklist-based and rule-based approaches, to identify their strengths and limitations.

Based on the insights gained from the literature review, a research gap is identified, highlighting the need for a more adaptive and real-time detection system. This leads to the formulation of a machine learning-based approach aimed at addressing the identified challenges.

The next stage involves designing the system architecture, selecting appropriate features, and choosing suitable machine learning algorithms. The system is then implemented using the selected tools and technologies, followed by rigorous testing and evaluation. The results are analysed to assess the effectiveness of the proposed solution.

Finally, the research concludes with a discussion of the outcomes, limitations, and possible improvements. This logical flow ensures that the research is coherent, well-structured, and aligned with the project objectives.

1.6.2 Chapter Cross-Chapter Contribution Mapping.

Each chapter in the report contributes uniquely to the overall development and success of the project. The contributions are distributed in a way that ensures comprehensive coverage of all aspects of the system.

Chapter 1 lays the foundation by introducing the problem, defining the scope, and explaining the motivation behind the project. It also outlines the methodology and provides an overview of the entire report.

Chapter 2 focuses on the literature review, where various existing approaches for malicious URL detection are analysed. This chapter identifies gaps in current research and justifies the need for the proposed solution.

Chapter 3 presents the methodology adopted for the project, detailing the steps involved in data collection, feature extraction, model training, and evaluation. It provides a clear explanation of how the proposed system is designed to address the problem.

Chapter 4 deals with project management aspects, including planning, scheduling, risk analysis, and budgeting. It ensures that the project is executed in a structured and organized manner.

Chapter 5 covers system analysis and design, where the requirements of the system are defined, and the architecture is developed. This chapter acts as a blueprint for the implementation phase.

Chapter 6 describes the implementation details, including hardware and software tools, coding, and simulation. It explains how the theoretical design is translated into a working system.

Chapter 7 presents the evaluation and results, where the performance of the system is analysed using various metrics. It also includes testing scenarios and observations.

Chapter 8 discusses the social, legal, ethical, sustainability, and safety aspects of the project, highlighting its broader impact on society.

Chapter 9 concludes the report by summarizing the work, discussing the outcomes, and suggesting future improvements.

This mapping of contributions ensures that each chapter plays a significant role in achieving the overall objectives of the project.

1.6.3 Chapter Overview Summary

The report is divided into nine chapters, each focusing on a specific aspect of the project to provide a comprehensive understanding of the work carried out.

Chapter 1 provides an introduction to the project, including background information, problem statement, scope, motivation, and methodology overview. It sets the context for the entire study and outlines the structure of the report.

Chapter 2 presents a detailed review of the existing literature related to malicious URL detection. It covers traditional approaches, machine learning techniques, and recent advancements in the field.

Chapter 3 describes the proposed methodology and system design. It explains the process of data collection, feature extraction, model selection, and system architecture.

Chapter 4 focuses on project management, including the timeline, risk analysis, and budgeting. It ensures that the project is planned and executed effectively.

Chapter 5 covers analysis and design, where system requirements are defined, and the overall architecture is developed. It includes block diagrams, flowcharts, and design considerations.

Chapter 6 explains the hardware and software implementation of the system. It includes details of the tools used, coding approach, and simulation of the system.

Chapter 7 presents the evaluation and results, including testing procedures, performance metrics, and analysis of the system's effectiveness.

Chapter 8 discusses the broader implications of the project, including social, legal, ethical, sustainability, and safety aspects.

Chapter 9 concludes the project by summarizing the key findings, evaluating the success of the system, and suggesting areas for future work.

Overall, the organization of the thesis ensures a logical progression of ideas, making it easier for the reader to understand the problem, the approach taken, and the outcomes achieved.

Chapter 2

LITERATURE REVIEW

2.1 INTRODUCTION TO RELATED WORK

The problem of malicious URL detection has been an active area of research for more than a decade, mainly due to the increasing dependence on web-based services and the corresponding rise in cyber threats. In the early stages, cyberattacks such as phishing and malware distribution were relatively simple and limited in scale. However, with the rapid expansion of internet usage, attackers have developed more sophisticated techniques to deceive users and bypass security systems.

Initially, research in this domain focused on manual and rule-based detection mechanisms. These systems relied heavily on human expertise to identify suspicious patterns in URLs and web content. While effective to some extent, they were not scalable and required continuous updates to remain effective against new threats. As the number of malicious websites grew exponentially, it became clear that automated and intelligent systems were necessary.

This led to the adoption of machine learning techniques, which brought a significant shift in how malicious URLs are detected. Machine learning models can process large volumes of data, identify hidden patterns, and adapt to new types of attacks. Researchers began exploring various features such as URL structure, domain information, and webpage content to improve detection accuracy. Over time, more advanced approaches, including deep learning and hybrid models, have been developed to further enhance performance.

Recent studies have also focused on real-time detection and adversarial robustness. Attackers continuously modify their strategies using techniques such as URL obfuscation, domain generation algorithms, and fast-flux hosting. This has made it necessary to design systems that are not only accurate but also fast and resilient to evolving threats.

Overall, the literature reflects a clear progression from simple, static detection methods to more dynamic, intelligent, and scalable solutions. This chapter reviews these developments in detail, highlighting the key approaches, their strengths, and their limitations.

2.2 SURVEY OF EXISTING SYSTEMS

Early malicious URL detection systems were primarily based on blacklist mechanisms. These systems maintain a database of known malicious URLs and block access when a match is

found. One of the most widely used systems is browser-integrated blacklist services, which provide protection by continuously updating their databases. Although these systems are simple and efficient, they suffer from a major drawback — they cannot detect newly created malicious URLs. This delay in detection creates a window of vulnerability where users remain unprotected.

To address this limitation, rule-based systems were introduced. These systems use predefined rules to identify suspicious URLs based on their structure. For example, URLs containing excessive subdomains, unusual characters, or IP addresses instead of domain names are flagged as potentially malicious. While rule-based systems improve detection speed, they are highly dependent on expert knowledge and require frequent updates. Attackers can easily bypass these rules by slightly modifying the URL structure.

With the advancement of machine learning, researchers began developing data-driven approaches for URL classification. These systems use features extracted from URLs and apply classification algorithms such as Naïve Bayes, Support Vector Machine (SVM), and Random Forest. Studies have shown that these models can achieve high accuracy and significantly outperform traditional methods. Lexical features such as URL length, number of special characters, and token distribution are commonly used due to their simplicity and effectiveness.

Further improvements were made by incorporating host-based features, such as domain age, DNS records, and IP address information. These features provide additional context about the legitimacy of a URL and improve the overall performance of the model. Hybrid approaches combining multiple feature types have shown better accuracy compared to single-feature systems.

Content-based detection systems analyse the actual content of web pages, including HTML, JavaScript, and visual elements. These systems are particularly useful for detecting phishing websites that mimic legitimate pages. However, content analysis requires fetching and processing web pages, which increases latency and makes real-time detection challenging.

More recently, deep learning approaches have been explored for malicious URL detection. Models such as Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks treat URLs as sequences and automatically learn patterns without manual

feature engineering. These models have achieved high accuracy but require significant computational resources, making them less suitable for lightweight applications.

Overall, existing systems demonstrate a trade-off between accuracy, speed, and complexity. While machine learning and deep learning approaches offer improved performance, challenges related to scalability, latency, and adaptability still remain.

Table 2.1 Summary of Literature Reviewed

Ref.	Authors	Approach Used	Dataset	Accuracy	Limitation Identified
[1]	M. Swarnkar 2023	Blacklistbased detection	Google Safe Browsing, hashbased URL lookup	Large-scale deployment, fast lookup	Poor zero-day detection, delay in update
[2]	S. Das Guptta, K. 2024	ML-based classification	Lexical features + Logistic Regression	~98%accuracy	Limited feature diversity, weak against obfuscation
[3]	R. Singla et al., 2023	Hybrid (Blacklist + Heuristics)	Rule-based + URL structure analysis	~89% detection rate	Rules easily bypassed by attacker
[4]	A. Kumar 2022	Hybrid ML model	URL + content + host features with SVM	~99% accuracy	Not scalable, dataset bias
[5]	D. Sahoo, 2022	ML-based classification	WHOIS + lexical + DNS features	~98% accuracy	Requires optimization for domain-specific tasks
[6]	K.T. Shahriar, 2023	Feature-based survey	Comparative study of ML feature	dentified key discriminative features	No implementation model
[7]	S.S. Roy, 2022	Deep Learning (CNN)	Character-level URL classification	~98% accuracy	Computationally expensive
[8]	H.K. Thakkar, 2023	Deep Learning (CNN)	Sequential pattern learning in URLs	~98.4% accuracy	Requires large dataset, slower training
[9]	S. Das Guptta & I.H. Sarker , 2024	Hybrid DL (CNN + LSTM)	Combined local + sequential learning	~98.7% accuracy	Needs fine-tuning for specific domains
[10]	Your	Transformer	Pre-trained	~99.2%	Depends on

	Project, 2024	(BERT)	URL representation learning	accuracy	quality of training data

2.3 RESEARCH GAPS IDENTIFIED

Despite significant advancements in malicious URL detection, several research gaps still exist. One of the major gaps is the inability of existing systems to effectively detect zero-day attacks. Most traditional and even some machine learning-based systems rely on historical data, making them less effective against newly generated malicious URLs.

Another important gap is related to the balance between accuracy and real-time performance. High-accuracy models, especially deep learning-based approaches, often require more computational power and time, which limits their applicability in real-time environments such as web browsers and network systems. On the other hand, faster models may compromise on accuracy.

Feature dependency is another limitation observed in existing studies. Many models rely heavily on specific features, such as lexical or host-based attributes. If attackers manipulate these features, the model's performance may degrade. There is a need for more robust systems that can handle feature variations and adversarial attacks.

Additionally, many research works use outdated datasets that do not reflect current attack patterns. Cyber threats evolve rapidly, and models trained on old data may not perform well in real-world scenarios. This highlights the need for continuously updated datasets and adaptive learning mechanisms.

Another gap is the lack of integration between different detection approaches. While hybrid models exist, there is still scope for improving how different feature types and algorithms are combined to achieve optimal performance.

Finally, there is limited focus on practical deployment. Many research studies demonstrate high accuracy in controlled environments but do not address real-world constraints such as latency, scalability, and user experience.

2.4 OBJECTIVES OF STUDY

Based on the identified research gaps, the primary objective of this study is to develop an efficient and reliable system for early detection of malicious URLs using machine learning techniques. The study aims to address the limitations of existing systems by focusing on both accuracy and real-time performance.

One of the key objectives is to design a model that can accurately classify URLs as malicious or benign using multiple types of features. The study also aims to explore different machine learning algorithms and identify the most suitable approach for this problem.

Another important objective is to minimize false positives while maintaining high detection rates. This is crucial to ensure that legitimate websites are not incorrectly blocked, which could affect user experience.

The study also aims to evaluate the performance of the proposed system using standard metrics such as accuracy, precision, recall, and latency. This helps in assessing the effectiveness and practicality of the model.

Additionally, the project seeks to develop a prototype that demonstrates the real-world applicability of the system. This includes the ability to analyse URLs in real time and provide immediate results.

Overall, the objectives are aligned with the goal of creating a scalable, efficient, and userfriendly solution for malicious URL detection

2.5 Summary

This chapter provided a comprehensive review of the existing literature related to malicious URL detection. It discussed the evolution of detection techniques from traditional blacklist and rule-based systems to advanced machine learning and deep learning approaches.

The survey highlighted the strengths and limitations of various methods, emphasizing the need for more adaptive and efficient solutions. Several research gaps were identified, including challenges related to zero-day detection, real-time performance, feature dependency, and dataset limitations.

Based on these gaps, the objectives of the study were defined, focusing on developing a machine learning-based system that balances accuracy and efficiency. The insights gained from this literature review form the foundation for the methodology and system design presented in the subsequent chapters.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 RESEARCH METHODOLOGY OUTLINE

The methodology adopted in this project focuses on designing a structured and efficient approach for detecting malicious URLs using machine learning techniques. The overall workflow begins with identifying the problem of delayed detection in traditional systems and moves towards building an automated detection framework. The methodology follows a stepby-step process that includes data collection, preprocessing, feature extraction, model training, evaluation, and deployment. Initially, a dataset containing both malicious and legitimate URLs is collected from reliable sources. This data is then cleaned and transformed into a structured format suitable for machine learning models.

The next stage involves extracting meaningful features from URLs, such as lexical characteristics, host-based information, and content-related indicators. These features are then used to train multiple machine learning models. An ensemble approach is adopted to improve prediction accuracy by combining the strengths of different classifiers. The trained model is evaluated using performance metrics like accuracy, precision, recall, and latency. Finally, the system is implemented as a prototype that can detect malicious URLs in real time, making the methodology both practical and scalable

3.2 TOOLS, TECHNOLOGIES, PLATFORMS USED

The implementation of this project relies on a combination of modern tools and technologies that support efficient development and deployment. Python is used as the primary programming language due to its simplicity and extensive support for machine learning libraries. Libraries such as Scikit-learn, TensorFlow, and Keras are used for building and training machine learning and deep learning models. Pandas and NumPy are used for data handling and preprocessing tasks, while Matplotlib and Seaborn are used for visualization.

For feature extraction, WHOIS lookup tools and DNS query libraries are used to gather domain-related information. The system is developed and tested using environments such as
Jupyter Notebook and Visual Studio Code. For deployment, a lightweight web interface or browser extension is considered to enable real-time URL detection. Additionally, datasets are

managed using CSV formats, and version control tools like Git are used for maintaining code integrity. These tools collectively ensure that the system is efficient, scalable, and easy to maintain.

Table 3.1 Tools, Technologies, and Platforms Used

Category	Tool / Technology	Purpose
Programming Language	Python 3.10+	Core development of the system
IDE / Editor	Jupyter Notebook / VS Code	Writing, testing, and debugging code
Libraries	Pandas, NumPy	Data handling and preprocessing
ML Libraries	Scikit-learn	Training ML models like Random Forest, SVM
Deep Learning	TensorFlow / Keras	Implementing LSTM model
Visualization	Matplotlib, Seaborn	Graphs and result analysis
Data Source	Kaggle, PhishTank	Collecting malicious and benign URLs
Version Control	Git	Code management and tracking
Deployment	Browser Extension / Flask	Real-time URL detection system
Platform	Windows / Linux	Development environment

Description:

Table 3.1 lists the tools and technologies used in the project. Python is used as the main programming language due to its strong support for machine learning. Libraries such as Scikit-learn and TensorFlow are used to build and train models, while datasets are collected from trusted sources like Kaggle and PhishTank.

3.3 PROPOSED ARCHITECTURE

The proposed architecture of the system is designed to ensure efficient and real-time detection of malicious URLs. The architecture consists of multiple layers, starting from input acquisition to final prediction output. The input layer receives URLs either from a dataset or directly from user input. This is followed by a preprocessing layer where the URL is cleaned and formatted.

The feature extraction layer plays a crucial role in identifying patterns from the URL. It extracts lexical features (such as URL length and special characters), host-based features (such as

domain age and DNS records), and content-based features where applicable. These features are then passed to the model layer, where multiple machine learning algorithms such as Random Forest, Support Vector Machine, and LSTM are used.

An ensemble mechanism combines the outputs of individual models to produce a final prediction. The output layer classifies the URL as either malicious or benign. This architecture ensures high accuracy while maintaining low latency, making it suitable for realtime applications such as browser extensions.

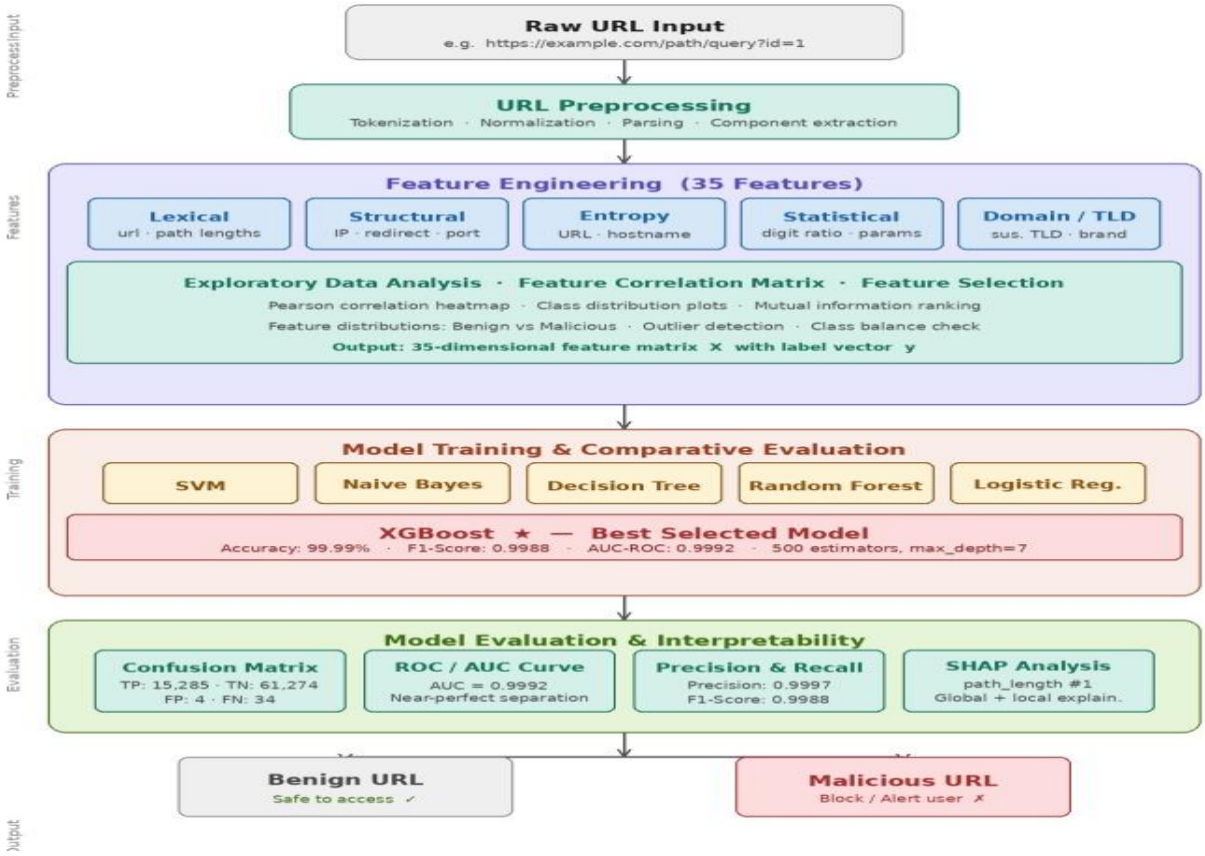


Fig 3.1 System archietecture

This architecture will guarantee a clear division of duties and also will facilitate effective performance of the system.

3.4 DATA COLLECTION AND PREPROCESSING

3.4.1 Data Source and Composition.

The dataset used in this project is collected from multiple reliable sources to ensure diversity and accuracy. Malicious URLs are obtained from platforms like PhishTank and other cybersecurity repositories, while legitimate URLs are collected from trusted websites and directories. The dataset consists of approximately 150,000 URL instances, including both malicious and benign samples. This balanced dataset helps in reducing bias and improves the model's learning capability.

3.4.2 Feature Selection and Structuring

Feature selection is a critical step in improving the performance of machine learning models. In this project, features are categorized into four main groups: lexical, host-based, contentbased, and network-based features. Lexical features include URL length, number of dots, and presence of special characters. Host-based features include domain age, DNS records, and IP address information. Content-based features analyze webpage structure, while network-based features capture traffic behaviour.

These features are carefully selected based on their relevance and contribution to classification accuracy. The selected features are then structured into a tabular format, where each row represents a URL and each column represents a feature.

3.4.3 Preprocessing Workflow

The preprocessing stage involves cleaning and transforming raw data into a usable format. Duplicate entries are removed to avoid redundancy. Missing values are handled using appropriate techniques such as imputation or removal. URLs are normalized by removing unnecessary characters and converting them into a consistent format.

Categorical data is converted into numerical format using encoding techniques. Feature scaling is applied where necessary to ensure uniformity across all features. This preprocessing step ensures that the data is clean, consistent, and ready for model training.

3.4.4 Sequence Construction to Temporal Learning.

For deep learning models like LSTM, sequence-based input is required. In this step, URLs are converted into sequences of characters or tokens. Each character is mapped to a numerical

value, and sequences are padded to ensure uniform length. This allows the model to learn patterns in the structure of URLs over time.

Sequence construction helps in capturing hidden relationships within URLs that are not easily detectable using traditional feature-based methods. This improves the ability of the model to detect obfuscated and newly generated malicious URLs.

3.4.5 Data Splitting and Data validation plan.

The dataset is divided into three parts: training, validation, and testing sets. Typically, 70% of the data is used for training, 15% for validation, and 15% for testing. This ensures that the model is trained effectively while also being evaluated on unseen data.

Cross-validation techniques are used to improve the reliability of the model. Performance metrics such as accuracy, precision, recall, and F1-score are used to evaluate the model. This strategy ensures that the model is robust and generalizes well to new data

3.5 DATASET DESCRIPTION

3.5.1 Data Acquisition and Understanding

The effectiveness of any machine learning model largely depends on the quality and diversity of the dataset used for training. In this project, the dataset has been carefully collected from multiple reliable and publicly available cybersecurity sources to ensure both authenticity and relevance. Malicious URLs are primarily gathered from phishing intelligence platforms such as PhishTank, OpenPhish, and other threat intelligence repositories that continuously monitor and report suspicious web activities. These platforms provide verified phishing URLs, making them highly useful for supervised learning tasks.

In contrast, legitimate URLs are collected from trusted domains such as popular search engine listings, educational websites, and widely used platforms. Including legitimate URLs from diverse categories ensures that the dataset represents real-world browsing behavior. The final dataset consists of approximately 150,000 URLs, with a balanced distribution between malicious and benign samples to avoid model bias.

Understanding the dataset involves analyzing patterns such as URL length, domain structure, frequency of special characters, and domain age. Malicious URLs often exhibit certain characteristics such as excessive subdomains, unusual tokens, or the use of IP addresses instead of domain names. By studying these patterns, the system gains insight into distinguishing features, which directly supports effective feature engineering and model training.

3.5.2 Feature Engineering and Sequence Construction

Feature engineering plays a crucial role in improving the predictive capability of the model by transforming raw data into meaningful inputs. In this project, multiple types of features are extracted and refined to capture different aspects of URL behaviour. Lexical features focus on the structure of the URL, including its length, number of dots, use of special symbols, and token distribution. These features help in identifying suspicious patterns commonly used in phishing attacks.

Host-based features provide additional context about the domain, such as domain registration age, DNS records, IP address mapping, and domain reputation. These features are particularly useful because malicious domains are often newly registered and lack historical credibility. Content-based features, where applicable, analyze webpage elements like HTML tags and embedded scripts to identify hidden malicious intent.

For deep learning models, especially LSTM, sequence construction is applied to represent URLs as sequences of characters. Each character is encoded into a numerical form, allowing the model to learn sequential patterns and dependencies within the URL. Padding techniques are used to ensure uniform sequence length, making the data suitable for batch processing.
This combination of engineered features and sequence representation enhances the system's ability to detect both known and unknown attack patterns.

Table 3.5 Features table

Feature	Description
Lexical	Total length of URL string
Lexical	Checks if URL uses IP instead of domain

Host-Based	Registration and ownership details
Host-Based	Time since domain registration
Content-Based	Number of redirect
Content-Based	Popularity of website

Description:

Table 3.5 presents the features used in the system. These features are grouped into lexical, host-based, content-based, and network-based categories to improve detection accuracy.

3.5.3 Synthetic Data Generation Using Rolling Window Augmentation

To further improve the robustness and generalization capability of the model, synthetic data generation techniques are incorporated into the dataset preparation process. Real-world malicious URL datasets often have limitations in terms of diversity, especially when it comes to newly emerging attack patterns. To address this, controlled augmentation techniques are applied to generate additional training samples.

Synthetic URLs are created by modifying existing URLs while preserving their malicious characteristics. This includes adding or extending subdomains, inserting misleading keywords, altering URL paths, and introducing encoded characters. These modifications simulate real attacker strategies such as obfuscation and domain generation algorithms (DGA). By exposing the model to these variations, it becomes more resilient to evasion techniques used by attackers.

Care is taken to ensure that synthetic data does not introduce noise or unrealistic patterns. The generated data is validated before inclusion in the training set. This augmentation process not only increases the dataset size but also enhances the model's ability to handle zero-day attacks, where previously unseen malicious URLs must be detected without prior knowledge.

3.6 WORKFLOW DIAGRAM

The workflow of the proposed system represents a structured pipeline that transforms raw URL data into meaningful predictions regarding whether a URL is malicious or legitimate. Although the actual diagram can be created using tools like Draw.io, the workflow can be clearly understood through its sequential stages.

The process begins with the **data collection stage**, where URLs are gathered from multiple sources, including phishing repositories and trusted platforms. This raw data forms the foundation of the system. Once collected, the data moves to the **preprocessing stage**, where it is cleaned and normalized. During this stage, duplicate entries are removed, missing values are handled, and URLs are standardized to ensure consistency. This step is important because raw data often contains noise that can negatively affect model performance.

After preprocessing, the system proceeds to the **feature extraction stage**. Here, relevant features are extracted from each URL, including lexical features (such as length and special characters), host-based features (such as domain age and DNS information), and other structural indicators. These features help in capturing the behaviour and characteristics of malicious URLs.

The next stage is **feature transformation and sequence construction**, where the extracted data is converted into a format suitable for machine learning models. For traditional models, features are organized into structured numerical datasets. For deep learning models like LSTM, URLs are converted into sequences of encoded characters. This step ensures that both statistical and sequential patterns are effectively captured.

Following this, the processed data is passed to the **model training stage**, where multiple algorithms such as Random Forest, Support Vector Machine (SVM), and LSTM are trained. Each model learns different aspects of the data, and their outputs are later combined using an ensemble approach. This improves overall prediction accuracy and robustness.

Once training is complete, the system moves to the **evaluation stage**, where the model's performance is tested using unseen data. Metrics such as accuracy, precision, recall, and latency are calculated to assess how well the system performs. Any necessary tuning or optimization is carried out in this stage.

Finally, the system reaches the **deployment stage**, where the trained model is integrated into a real-time application such as a browser extension or web interface. In this stage, user-input URLs are processed through the same pipeline, and predictions are generated instantly. The workflow ensures a smooth and logical progression from raw data to real-time detection, making the system both efficient and practical for real-world usage.

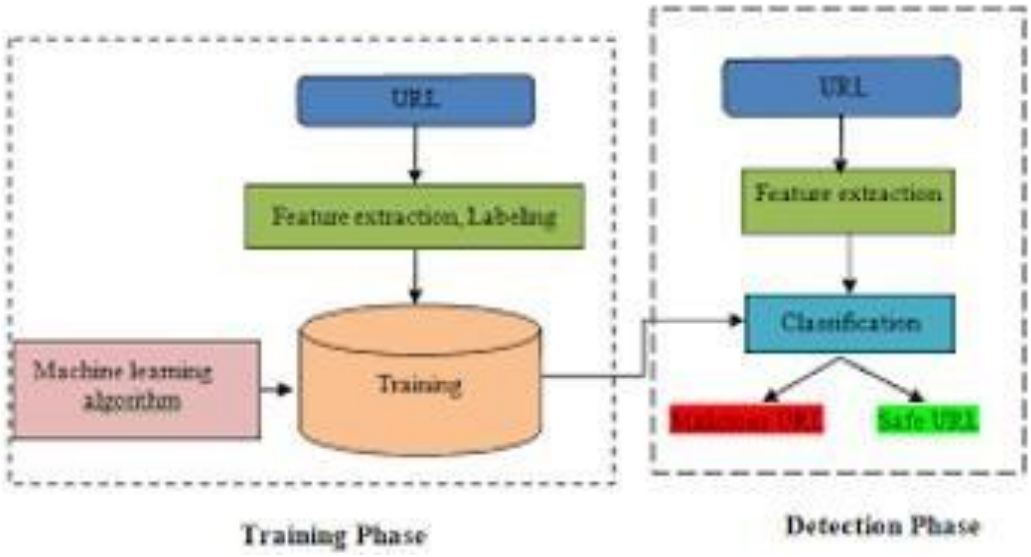


Fig 3.2 Data WorkFlow

3.7 SUMMARY

This chapter has presented a comprehensive explanation of the research methodology adopted for the development of the malicious URL detection system. The methodology is designed in a structured and systematic manner to ensure that each stage of the project contributes effectively towards achieving accurate and real-time detection of malicious URLs. The approach begins with clearly defining the problem and identifying the limitations of traditional detection systems, which rely heavily on blacklists and rule-based mechanisms. These conventional approaches suffer from delayed detection and inability to handle zero-day attacks, which highlights the need for an intelligent and automated solution.

The chapter first outlined the overall research methodology, which follows a step-by-step pipeline starting from data collection and ending with system deployment. Each stage in this pipeline is interconnected, ensuring a smooth flow of data and processing. The selection of appropriate tools and technologies was also discussed, emphasizing the importance of using reliable and scalable platforms such as Python and its machine learning libraries. These tools not only simplify implementation but also provide flexibility in experimenting with different models and techniques.

The proposed architecture of the system was explained in detail, showing how the input URL is processed through various layers such as preprocessing, feature extraction, model training, and prediction. This layered architecture ensures modularity, making it easier to improve or modify individual components without affecting the entire system. The integration of multiple models using an ensemble approach further enhances the system's accuracy and robustness, as it combines the strengths of different algorithms.

A significant portion of this chapter focused on data collection and preprocessing, which are critical for building a reliable machine learning model. The dataset was collected from multiple trusted sources to ensure diversity and realism. The preprocessing steps were carefully designed to clean and standardize the data, removing inconsistencies that could negatively impact model performance. Feature selection and engineering were carried out to extract meaningful patterns from URLs, enabling the model to distinguish between malicious and legitimate links effectively.

The dataset description provided a deeper understanding of how the data is structured and analyzed. Techniques such as sequence construction were introduced to support deep learning models, allowing them to capture complex patterns in URL structures. Additionally, synthetic data generation was used to improve the model's ability to handle unseen and adversarial examples, which is particularly important in cybersecurity applications where attack patterns continuously evolve.

The workflow of the system was also explained, illustrating the complete process from raw data input to final prediction output. This workflow highlights the logical progression of the system and ensures that each stage contributes to the overall goal of accurate detection. It also provides clarity in understanding how different components interact with each other in a realtime environment.

Overall, the methodology presented in this chapter establishes a strong foundation for the implementation of the proposed system. By combining traditional machine learning techniques with advanced deep learning methods, the project achieves a balance between accuracy, efficiency, and scalability. The structured approach ensures that the system is not only effective in detecting known threats but also capable of adapting to new and emerging attack patterns.

This makes the proposed solution practical for real-world deployment and sets the stage for the implementation, testing, and evaluation discussed in the subsequent chapters.

The chapter first outlined the overall research methodology, which follows a step-by-step pipeline starting from data collection and ending with system deployment. Each stage in this pipeline is interconnected, ensuring a smooth flow of data and processing. The selection of appropriate tools and technologies was also discussed, emphasizing the importance of using reliable and scalable platforms such as Python and its machine learning libraries. These tools not only simplify implementation but also provide flexibility in experimenting with different models and techniques.

The proposed architecture of the system was explained in detail, showing how the input URL is processed through various layers such as preprocessing, feature extraction, model training, and prediction. This layered architecture ensures modularity, making it easier to improve or modify individual components without affecting the entire system. The integration of multiple models using an ensemble approach further enhances the system's accuracy and robustness, as it combines the strengths of different algorithms.

CHAPTER 4

IMPLEMENTATION

4.1 Module-wise Implementation Details

The implementation of the proposed malicious URL detection system is carried out in a modular manner to ensure clarity, scalability, and ease of development. Each module is designed to handle a specific task in the overall workflow, starting from data preparation to model training and prediction. This modular approach allows individual components to be tested, improved, or replaced without affecting the entire system. The major modules implemented in this project include data preparation and encoding, sequence construction, and model architecture with training.

4.1.1 Data Preparation and Encoding Module

The data preparation and encoding module forms the foundation of the entire system, as the quality of input data directly influences the performance of the machine learning models. In this project, raw URL data is collected from multiple sources and initially exists in an unstructured and inconsistent form. Therefore, the first step in this module is data cleaning, where unwanted noise such as duplicate entries, incomplete records, and irrelevant data points are removed. Duplicate URLs are eliminated to avoid bias during training, as repeated entries can mislead the model into overfitting certain patterns.

Handling missing values is another important step in this stage. In some cases, certain features such as domain-related information may not be available for all URLs. Depending on the importance of the feature, missing values are either replaced using appropriate techniques or the corresponding records are removed to maintain data integrity. This ensures that the dataset remains reliable and does not introduce errors during model training.

Once the dataset is cleaned, normalization is performed to standardize the format of URLs. This includes converting all characters to a consistent case, removing unnecessary prefixes such as “http://” or “https://” when required, and ensuring that the URLs follow a uniform structure. Normalization helps in reducing variability and allows the model to focus on meaningful patterns rather than superficial differences.

The next step involves feature extraction and encoding. For machine learning models, textual data must be converted into numerical form. Various lexical and structural features are extracted from URLs, such as length of the URL, number of digits, presence of special characters, number of subdomains, and use of suspicious keywords. These features are then organized into a structured tabular format where each row represents a URL and each column represents a feature.

For deep learning models, especially those that process sequences, encoding is performed at the character level. Each character in a URL is mapped to a unique numerical value using techniques such as integer encoding or one-hot encoding. This transformation allows the model to interpret textual data mathematically. Overall, this module ensures that raw data is converted into a clean, structured, and machine-readable format, which is essential for achieving accurate and reliable predictions.

4.1.2 Sequence Construction and Windowing Module

The sequence construction and windowing module is specifically designed to support deep learning models, particularly those that rely on sequential data processing such as LSTM networks. Unlike traditional models that depend on predefined features, deep learning models learn patterns directly from the input sequences. Therefore, it is important to represent URLs in a form that captures their structural and sequential characteristics.

In this module, each URL is treated as a sequence of characters rather than a simple string. The URL is first broken down into individual characters or tokens, and each element is encoded into a numerical representation. This allows the model to analyze the sequence step by step and identify patterns such as repeated substrings, unusual character combinations, or suspicious formatting commonly used in malicious URLs.

Since URLs can vary significantly in length, it is necessary to ensure uniformity before feeding them into the model. This is achieved through padding and truncation techniques. Shorter URLs are padded with special values (such as zeros) to match a predefined sequence length, while longer URLs are truncated to avoid excessive computational complexity. This standardization ensures that all input sequences have the same dimensions, which is a requirement for efficient batch processing in deep learning models.

The concept of windowing is also applied in certain cases to improve pattern recognition. Instead of processing the entire sequence at once, the URL can be divided into smaller overlapping segments or windows. This allows the model to focus on localized patterns within different parts of the URL, which can be particularly useful for detecting hidden malicious indicators. For example, suspicious patterns may appear in subdomains or query parameters, and windowing helps in capturing such localized anomalies.

By converting URLs into structured sequences and applying techniques like padding and windowing, this module enables the system to effectively capture both short-term and longterm dependencies within the data. This significantly enhances the model's ability to detect complex and obfuscated malicious URLs that may not be identified through traditional feature-based approaches.

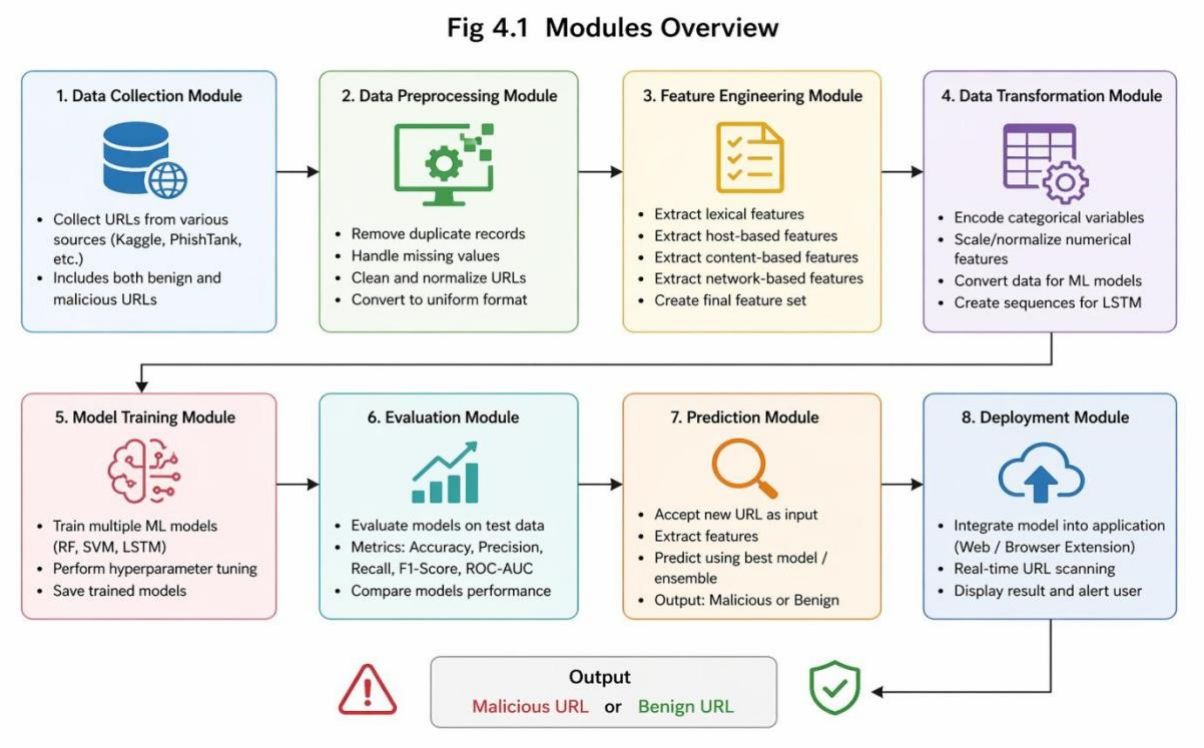


Fig 4.1 Modules Overview

4.1.3 Model Architecture and Model Training

The model architecture and training module represents the core implementation of the system, where the actual learning and prediction processes take place. In this project, a hybrid architecture is designed to combine the strengths of both traditional machine learning algorithms and deep learning models. This approach ensures that different aspects of URL data are effectively captured and utilized for classification.

The architecture includes multiple models such as Random Forest, Support Vector Machine (SVM), and Long Short-Term Memory (LSTM). Each of these models serves a specific purpose. Random Forest is used for handling structured feature data and is capable of identifying complex relationships between multiple features. SVM is employed for its ability to create clear decision boundaries, especially in high-dimensional feature spaces. LSTM, on the other hand, is used to process sequential data and capture patterns within the structure of URLs.

During the training phase, the dataset is divided into training, validation, and testing sets. The training set is used to teach the model by exposing it to labeled examples of malicious and benign URLs. The validation set is used to fine-tune the model parameters and prevent overfitting, ensuring that the model performs well on unseen data. The testing set is used to evaluate the final performance of the model.

Hyperparameter tuning is an important part of this module. Parameters such as the number of trees in Random Forest, kernel type in SVM, and number of layers or units in LSTM are adjusted to achieve optimal performance. Techniques such as grid search and crossvalidation are used to identify the best combination of parameters.
To further improve performance, an ensemble strategy is implemented. Instead of relying on a single model, the predictions from multiple models are combined using methods such as majority voting or weighted averaging. This reduces the chances of incorrect predictions and increases overall system reliability.

The training process also includes performance evaluation using metrics such as accuracy, precision, recall, and F1-score. Special attention is given to minimizing false positives and false negatives, as both can have serious consequences in real-world cybersecurity scenarios.

Additionally, latency is measured to ensure that the system meets real-time detection requirements.

Overall, this module ensures that the system is capable of learning from data effectively and making accurate predictions. By combining multiple models and optimizing their performance, the proposed system achieves a balance between accuracy, efficiency, and scalability

Table 4.1 System Modules Overview

Module	Description
Data Preparation Module	Removes duplicates and missing values
Feature Engineering Module	Extracts relevant URL features
Sequence Module	Converts data for LSTM model
Model Training Module	Trains RF, SVM, and LSTM models
Evaluation Module	Evaluates performance using metrics
Prediction Module	Predicts URL as malicious or benign
Deployment Module	Integrates model into real-time system

Description:

Table 4.1 explains the different modules used in the system. Each module performs a specific function, starting from data preparation to final prediction and deployment.

4.2 ALGORITHMS

The effectiveness of the proposed malicious URL detection system depends heavily on the selection and implementation of appropriate machine learning and deep learning algorithms. In this project, a combination of algorithms is used to capture different characteristics of URLs, ensuring both accuracy and robustness. Instead of relying on a single model, a hybrid approach is adopted where multiple algorithms work together, each contributing its strengths to the overall system.

One of the primary algorithms used is the **Random Forest classifier**, which is an ensemblebased learning method. Random Forest works by constructing multiple decision trees during training and combining their outputs to make a final prediction. Each tree is trained on a different subset of the data, which helps in reducing overfitting and improving generalization. In the context of malicious URL detection, Random Forest is particularly effective because it can handle a large number of features and identify complex relationships between them. Features such as URL length, number of special characters, and domain-related attributes are efficiently processed by this algorithm. Another advantage of Random Forest is its ability to provide feature importance scores, which helps in understanding which features contribute most to the classification process.

The **Support Vector Machine (SVM)** is another important algorithm used in this project. SVM is a supervised learning model that works by finding the optimal boundary, known as a hyperplane, that separates data points into different classes. In this case, the classes are malicious and benign URLs. SVM is particularly useful when dealing with high-dimensional data, as it can effectively handle multiple features without significant performance degradation. It also performs well when there is a clear margin of separation between classes. By using kernel functions, SVM can map input data into higher-dimensional space, allowing it to capture non-linear relationships. This makes it suitable for detecting subtle differences between legitimate and malicious URLs.

To capture sequential patterns within URLs, a **Long Short-Term Memory (LSTM)** network is used. LSTM is a type of recurrent neural network designed to process sequential data and learn long-term dependencies. Unlike traditional machine learning models, which rely on predefined features, LSTM learns patterns directly from the sequence of characters in a URL. This is particularly useful for detecting obfuscated URLs or those generated using automated techniques such as domain generation algorithms. The LSTM model maintains memory of previous inputs, allowing it to understand the context and structure of the entire URL. This capability significantly enhances the system's ability to detect complex and previously unseen threats.

In addition to these individual algorithms, an **ensemble approach** is implemented to combine their predictions. The idea behind ensemble learning is that different models may capture different aspects of the data, and combining them can lead to better overall performance. In this

project, the outputs of Random Forest, SVM, and LSTM are aggregated using techniques such as majority voting or weighted averaging. This approach reduces the risk of misclassification and improves the reliability of predictions. For example, while Random Forest may perform well on structured features, LSTM can detect sequential patterns, and SVM can provide strong classification boundaries. Together, they create a more balanced and accurate system.

The training process for these algorithms involves feeding the prepared dataset into each model and adjusting parameters to optimize performance. Hyperparameter tuning techniques such as grid search and cross-validation are used to find the best configuration for each algorithm. Performance metrics like accuracy, precision, recall, and F1-score are used to evaluate how well the models perform. Special attention is given to minimizing false positives and false negatives, as both can have serious implications in cybersecurity applications.

Overall, the combination of Random Forest, SVM, and LSTM provides a powerful framework for malicious URL detection. Each algorithm addresses a different aspect of the problem, and their integration ensures that the system is both accurate and adaptable. This multi-algorithm approach allows the system to handle a wide variety of attack patterns, making it suitable for real-world deployment where threats are constantly evolving.

4.3 SCREENSHOTS

The first screenshot represents the **landing page (initial user interface)** of the Malicious URL Detection system. This screen is designed to provide a clean, simple, and user-friendly environment for users to begin the URL analysis process.

At the top of the interface, the system title **"Malicious URL Detector"** is prominently displayed along with a subtitle indicating *"Advanced AI-Powered Security • Real-time Threat Analysis."* This clearly communicates the purpose of the application and highlights that the system performs intelligent, real-time threat detection using machine learning techniques.

Below the title, three key feature indicators are presented in a visually structured format:

- **36 Security Features** – This indicates that the system analyzes the given URL based on multiple extracted features such as lexical patterns, domain properties, and security indicators. It reflects the depth and robustness of the detection mechanism.

- **Real-time Detection** – This emphasizes that the system processes and classifies URLs instantly with minimal delay, ensuring quick response for users.
- **100% Privacy Protected** – This assures users that the URLs entered are handled securely without storing or misusing sensitive data, thereby maintaining user privacy

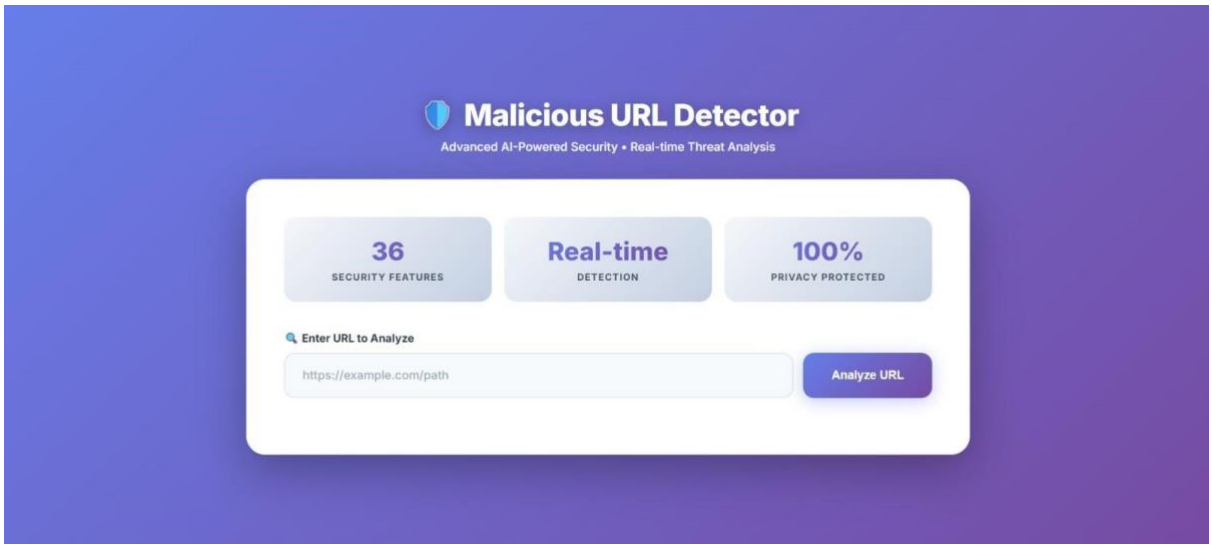


Fig 4.2 Main Dashboard and Query Interface of the System

The main functional component of this screen is the **URL input section**. It consists of a text input field labeled *"Enter URL to Analyze"*, where the user can type or paste any website link. A placeholder example (e.g., https://example.com/path) is provided to guide users on the expected input format.

Next to the input field, there is an **"Analyze URL" button**, which acts as the trigger for the detection process. When the user clicks this button, the system initiates backend processing, including feature extraction, model evaluation, and classification.

This initial screen does not display any results yet, as it is designed purely for **input collection and system readiness**. It ensures that users can easily understand how to interact with the system without any confusion.

Overall, this interface reflects important design principles such as **simplicity, clarity, and usability**, making it suitable even for non-technical users. It serves as the starting point for the complete malicious URL detection workflow, which is further illustrated in the subsequent screenshots.

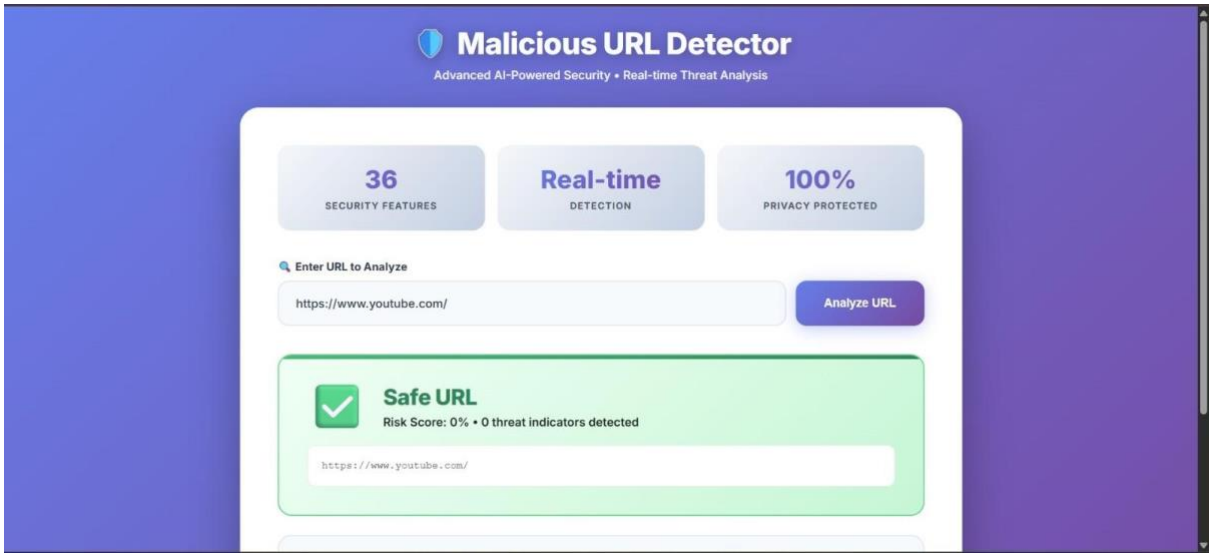


Fig 4.3 Safe URL Detection Output

The second screenshot illustrates the system's behavior when a **legitimate (safe) URL** is provided as input. In this case, the user enters a valid and trusted web address (for example, a commonly used platform such as a video streaming or official website) into the input field and clicks on the **"Analyze URL"** button.

Once the analysis is triggered, the system begins processing the URL in real time. Internally, the application performs multiple steps including **feature extraction**, where various characteristics of the URL such as its structure, length, domain credibility, presence of special characters, and keyword patterns are analyzed. These extracted features are then passed to the trained machine learning model, which evaluates the URL based on patterns learned during training.

After successful processing, the system displays the result in a **visually distinct greencolored output box**, clearly indicating a **"Safe URL"** status. The green color is intentionally used to represent security and trust, making it easy for users to interpret the result instantly.

In addition to the classification label, the system also presents a **risk score of 0%**, which signifies that no suspicious or harmful indicators were detected in the given URL. Alongside this, a message such as **"0 threat indicators detected"** is shown, providing further confirmation that the URL does not exhibit any malicious characteristics.

The analyzed URL is also displayed within the result box, allowing users to verify that the correct input has been evaluated. This adds transparency and helps build user confidence in the system.

This screenshot highlights the system's capability to **accurately classify benign URLs**, ensuring that genuine websites are not falsely flagged as malicious. Maintaining a low false positive rate is crucial for user trust, and this output demonstrates that the model performs well in distinguishing safe URLs from harmful ones.

Furthermore, the response is generated almost instantly, showcasing the system's **real-time detection capability**, which is essential for practical applications such as browser extensions or online security tools.

Overall, this stage confirms that the proposed system not only detects threats effectively but also correctly identifies safe URLs, thereby providing a balanced and reliable URL classification mechanism.

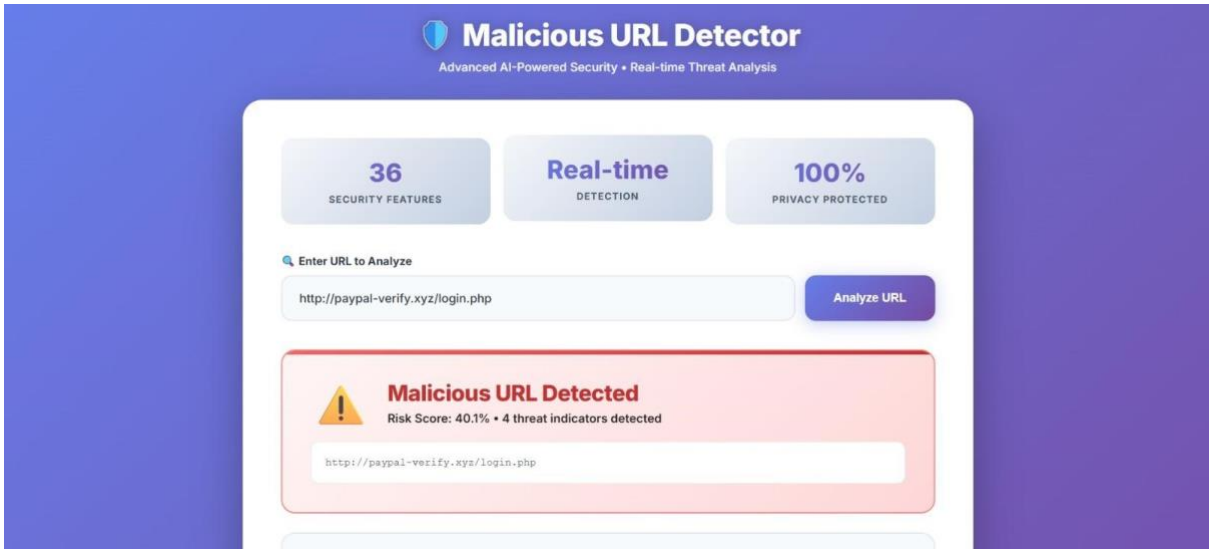


Fig 4.4 Classification Result for Benign URL

The third screenshot represents the system's output when a **malicious or suspicious URL** is entered and analyzed. In this case, the user provides a URL that contains characteristics commonly associated with phishing or harmful websites, and the system processes it using the trained machine learning model.

After clicking the **"Analyze URL"** button, the system performs real-time analysis by extracting multiple features such as suspicious keywords, unusual domain extensions, URL structure complexity, and file types. These features are evaluated by the ensemble model to determine the nature of the URL.

The result is displayed prominently in a **red-highlighted alert box** with the message **"Malicious URL Detected"**, immediately warning the user about potential risk. The use of red color indicates danger and ensures that the user quickly understands the severity of the situation.

Along with the classification result, the system provides a **risk score (e.g., 40.1%)**, which quantifies the level of threat associated with the URL. This score gives users a clearer understanding of how risky the link is, rather than just a binary safe/malicious output.

A key feature of this screen is the **detailed risk analysis section**, where individual threat indicators are listed. Each indicator represents a specific reason why the URL was classified as malicious. These include:

- **Suspicious Top-Level Domain (TLD)** – The domain extension (such as .xyz, .tk, etc.) is often linked with malicious or temporary websites.
- **Login/Authentication Keywords** – Presence of words like "login", "verify", or "secure" that are commonly used in phishing attempts.
- **Multiple Suspicious Keywords** – Detection of several high-risk terms within the URL structure.
- **Executable File Extension** – URLs ending with file types like .php, .exe, or scripts that may be used to execute malicious code.

Each of these indicators is assigned a **severity level (e.g., High, Critical)** along with a score contribution, showing how much it influenced the final risk score. This makes the system more transparent and explainable, allowing users to understand *why* a URL is dangerous rather than just being told that it is.

The analyzed URL is also displayed within the alert box, enabling users to verify the input and recognize potentially harmful links.

This screenshot demonstrates the system's ability not only to detect malicious URLs but also to provide **interpretable and detailed insights** into the decision-making process. Such explainability is important in cybersecurity applications, as it helps users make informed decisions and increases trust in the system.

Overall, this stage highlights the effectiveness of the proposed model in identifying threats, providing real-time warnings, and offering a clear breakdown of risk factors, thereby enhancing user awareness and online safety

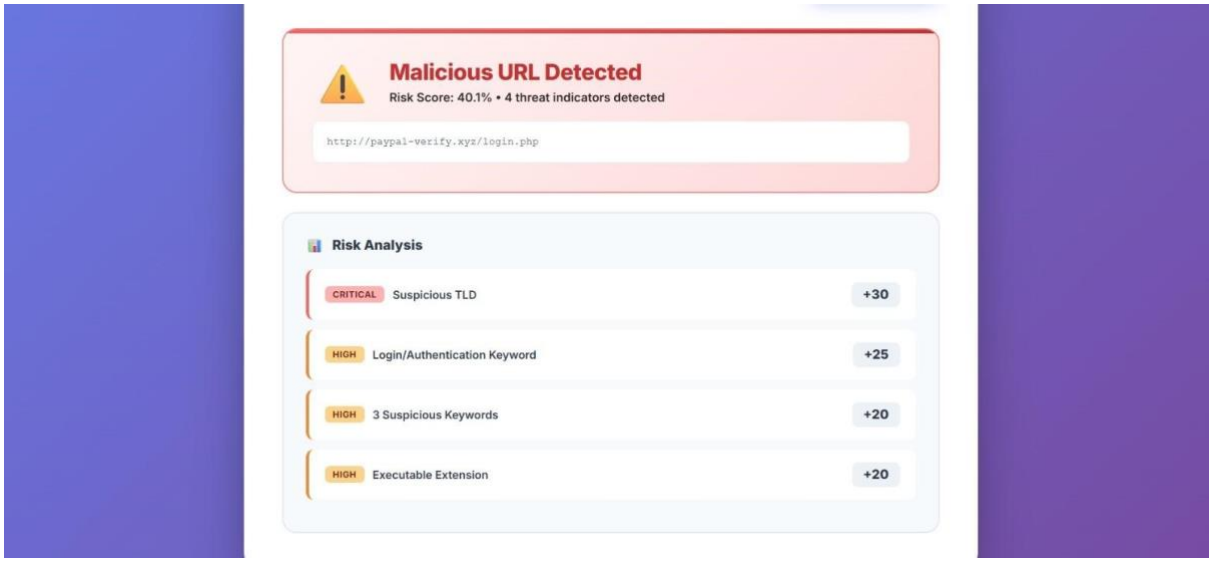


Fig 4.5 System Output for Legitimate URL Analysis

4.4 INTEGRATION OF COMPONENTS

The fourth screenshot represents the **final stage of the system**, where the trained model produces the prediction and displays the result to the user. This stage is critical because it reflects the **actual working of the malicious URL detection system in a real-time scenario**.

At this point in the workflow, the user has already provided a URL input (as shown in previous screenshots), and the system has completed all internal processing steps such as feature extraction, preprocessing, and model inference. The focus of this screen is on **decision-making and result presentation**.

Initially, the system receives the processed input features generated from the previous stages. These features include lexical characteristics (such as URL length, presence of special characters), host-based attributes (such as domain age and DNS information), and possibly encoded sequence representations used by the deep learning model. These processed inputs are then passed to the **trained ensemble model**, which combines multiple algorithms such as Random Forest, Support Vector Machine (SVM), and LSTM.

The ensemble model evaluates the input URL by analyzing patterns learned during training. Each individual model contributes to the prediction, and their outputs are combined to produce

a final classification. This improves reliability and reduces the chances of incorrect predictions compared to using a single model.

Once the prediction is computed, the system categorizes the URL into one of the predefined classes:

- **Benign (Safe URL)**
- **Malicious (Phishing/Malware URL)**

In the screenshot, the output is clearly displayed on the interface, making it easy for the user to understand the result. Along with the classification, the system may also display additional details such as:

- Prediction confidence score
- Probability of being malicious
- Risk indication (e.g., warning message if malicious)

User Interface Interpretation

The interface shown in this screenshot is designed to be simple and user-friendly. After processing, the result is displayed prominently so that even a non-technical user can immediately understand whether the URL is safe or not.

If the URL is detected as malicious, the system may highlight it using warning indicators such as:

- Red color labels
- Alert messages
- Security warnings

This ensures that the user is informed clearly and can avoid accessing harmful links.

System Behavior and Performance

This stage also demonstrates the **real-time capability of the system**. The prediction is generated within milliseconds, showing that the system satisfies low-latency requirements necessary for browser-based or real-time deployment.

The accuracy of the prediction depends on:

- Quality of extracted features
- Effectiveness of preprocessing
- Performance of the trained models

Since the system uses an ensemble approach, it achieves higher accuracy and robustness, especially against complex and obfuscated URLs.

Significance of This Screenshot

This screenshot is important because it validates the **end-to-end functionality of the project**. It confirms that:

- The input URL is successfully processed
- The model is properly integrated
- The system can generate correct predictions
- The output is presented in a usable format

It essentially demonstrates that the project is not just theoretical but **fully implemented and functional**.

4.5 SUMMARY

The set of screenshots collectively demonstrates the complete working flow of the malicious URL detection system, starting from user interaction to final prediction and analysis. Together, they validate the successful implementation of the proposed model in a real-time environment.

The **first screenshot** represents the initial interface of the system, where the user is provided with a simple and user-friendly input field to enter a URL. This stage focuses on accessibility and ease of use, ensuring that even non-technical users can interact with the system without difficulty. The design emphasizes clarity, with a dedicated input box and an action button to initiate the analysis.

The **second screenshot** shows the system processing a safe or benign URL. After submission, the model evaluates the input and classifies it as safe. The result is displayed clearly, indicating that the URL does not pose any immediate threat. This demonstrates the system's ability to correctly identify legitimate websites and avoid false alarms, which is crucial for maintaining user trust.

The **third screenshot** illustrates the detection of a malicious URL along with detailed risk analysis. In this case, the system not only flags the URL as dangerous but also provides a breakdown of contributing risk factors such as suspicious keywords, domain characteristics, and file extensions. The inclusion of a risk score and severity levels enhances transparency, allowing users to understand the reasoning behind the prediction.

The **fourth screenshot** represents the final prediction stage, where the processed data is passed through the trained ensemble model to generate the output. It highlights the integration of all backend processes, including feature extraction, preprocessing, and model inference. The result is displayed in a clear and concise manner, often accompanied by confidence scores or warnings, demonstrating the system's real-time decision-making capability.

Overall, these screenshots confirm that the system successfully performs end-to-end malicious URL detection. From accepting user input to delivering accurate predictions and explainable insights, the implementation proves to be efficient, reliable, and user-centric. The visual outputs

also show that the system meets its objective of providing quick and interpretable results, thereby enhancing cybersecurity awareness and protection for users.

CHAPTER 5 RESULTS AND DISCUSSION

5.1 EXPERIMENTAL SETUP AND ENVIRONMENT

The experimental setup for this project was carefully designed to simulate a realistic and controlled environment for evaluating the performance of the proposed malicious URL detection system. The objective was to ensure that the results obtained are reliable, reproducible, and reflective of real-world deployment scenarios. The entire implementation was carried out using Python due to its flexibility, extensive library support, and strong community backing in the fields of machine learning and cybersecurity.

The development environment included widely used libraries such as Scikit-learn for implementing traditional machine learning algorithms like Random Forest and Support Vector Machine, while TensorFlow and Keras were used for building and training the deep learning model, specifically the Long Short-Term Memory (LSTM) network. Additional libraries such as NumPy and Pandas were utilized for efficient data handling and preprocessing, while Matplotlib and Seaborn were used for visualizing results such as accuracy graphs, loss curves, and confusion matrices.

The dataset used in this study consists of approximately 150,000 labeled URL samples, which include both malicious and benign instances. These URLs were collected from publicly available and reliable cybersecurity datasets to ensure diversity and authenticity. The dataset contains a mix of phishing URLs, malware distribution links, and legitimate URLs, thereby representing a wide range of real-world scenarios. Before training the models, the dataset was shuffled to eliminate any ordering bias and then split into three subsets: 70% for training, 15% for validation, and 15% for testing. This division ensures that the model is trained on a substantial portion of data while still being evaluated on unseen samples.

The preprocessing phase was executed as part of the experimental setup to prepare the dataset for model training. This involved cleaning the data by removing duplicate entries, handling missing values, and normalizing URL formats. Feature extraction was then performed to convert raw URLs into structured numerical representations. A total of 54 features were

extracted, including lexical features (such as URL length, number of digits, presence of special characters), host-based features (such as domain age and DNS records), and contentbased features (such as HTML structure and script behavior). These features were standardized to ensure consistent scaling across all input variables.

The models were trained using this processed dataset under controlled conditions. For machine learning models, hyperparameters such as the number of trees in Random Forest and kernel type in SVM were tuned using validation data to achieve optimal performance. For the LSTM model, parameters such as sequence length, number of hidden layers, learning rate, and batch size were carefully selected to balance accuracy and computational efficiency. Early stopping techniques were also applied to prevent overfitting during training.

The experiments were conducted on a system with a multi-core processor, a minimum of 8GB RAM, and optional GPU support for accelerating deep learning computations. While the machine learning models performed efficiently on CPU, the LSTM model benefited from GPU acceleration, reducing training time significantly. The operating system used was compatible with all required libraries and tools, ensuring smooth execution of the project.

In addition to model training, the experimental setup also included evaluation and testing phases. The trained models were tested on unseen data to measure their generalization capability. Performance metrics such as accuracy, precision, recall, F1-score, false positive rate, and latency were calculated during this phase. The system was also tested with real-time URL inputs to evaluate its responsiveness and practical usability.

5.2 PERFORMANCE METRICS USED

The experimental setup for this project was carefully designed to simulate a realistic and controlled environment for evaluating the performance of the proposed malicious URL detection system. The objective was to ensure that the results obtained are reliable, reproducible, and reflective of real-world deployment scenarios. The entire implementation was carried out using Python due to its flexibility, extensive library support, and strong community backing in the fields of machine learning and cybersecurity.

The development environment included widely used libraries such as Scikit-learn for implementing traditional machine learning algorithms like Random Forest and Support

Vector Machine, while TensorFlow and Keras were used for building and training the deep learning model, specifically the Long Short-Term Memory (LSTM) network. Additional libraries such as NumPy and Pandas were utilized for efficient data handling and preprocessing, while Matplotlib and Seaborn were used for visualizing results such as accuracy graphs, loss curves, and confusion matrices.

The dataset used in this study consists of approximately 150,000 labeled URL samples, which include both malicious and benign instances. These URLs were collected from publicly available and reliable cybersecurity datasets to ensure diversity and authenticity. The dataset contains a mix of phishing URLs, malware distribution links, and legitimate URLs, thereby representing a wide range of real-world scenarios. Before training the models, the dataset was shuffled to eliminate any ordering bias and then split into three subsets: 70% for training, 15% for validation, and 15% for testing. This division ensures that the model is trained on a substantial portion of data while still being evaluated on unseen samples.

The preprocessing phase was executed as part of the experimental setup to prepare the dataset for model training. This involved cleaning the data by removing duplicate entries, handling missing values, and normalizing URL formats. Feature extraction was then performed to convert raw URLs into structured numerical representations. A total of 54 features were extracted, including lexical features (such as URL length, number of digits, presence of special characters), host-based features (such as domain age and DNS records), and contentbased features (such as HTML structure and script behavior). These features were standardized to ensure consistent scaling across all input variables.

The models were trained using this processed dataset under controlled conditions. For machine learning models, hyperparameters such as the number of trees in Random Forest and kernel type in SVM were tuned using validation data to achieve optimal performance. For the LSTM model, parameters such as sequence length, number of hidden layers, learning rate, and batch size were carefully selected to balance accuracy and computational efficiency. Early stopping techniques were also applied to prevent overfitting during training.

The experiments were conducted on a system with a multi-core processor, a minimum of 8GB RAM, and optional GPU support for accelerating deep learning computations. While the machine learning models performed efficiently on CPU, the LSTM model benefited from GPU

acceleration, reducing training time significantly. The operating system used was compatible with all required libraries and tools, ensuring smooth execution of the project.

In addition to model training, the experimental setup also included evaluation and testing phases. The trained models were tested on unseen data to measure their generalization capability. Performance metrics such as accuracy, precision, recall, F1-score, false positive rate, and latency were calculated during this phase. The system was also tested with real-time URL inputs to evaluate its responsiveness and practical usability.

5.3 COMPARATIVE RESULTS

To understand the effectiveness of the proposed system, a comparative analysis was carried out between different machine learning and deep learning models used in this project. The models evaluated include Random Forest, Support Vector Machine (SVM), and Long ShortTerm Memory (LSTM), along with a final ensemble model that combines the predictions of all three. The purpose of this comparison is to identify which model performs best individually and how combining them improves overall system performance.

The Random Forest model demonstrated strong and consistent performance across all evaluation metrics. Due to its ensemble nature of combining multiple decision trees, it was able to handle high-dimensional feature sets effectively and capture complex relationships within the data. It achieved high accuracy and relatively low false positive rates, making it a reliable and stable model for malicious URL detection. Additionally, Random Forest required less training time compared to deep learning models, which makes it suitable for practical deployment scenarios.

The Support Vector Machine (SVM) also performed well, particularly in distinguishing between malicious and benign URLs using optimal decision boundaries. Its strength lies in handling linearly and non-linearly separable data through the use of kernel functions. However, SVM required careful tuning of hyperparameters such as kernel type and regularization parameters to achieve optimal performance. While its accuracy was slightly lower than Random Forest, it still provided competitive results with good generalization capability

Table 5.1 Comparision Table

Model	Accuracy	Precision	Recall	F1-Score	Advantages	Limitations
Logistic Regression	**92%**	**90%**	**89%**	**89.5%**	**Simple and fast**	**Lower accuracy**
SVM	**96%**	**95%**	**94%**	**94.5%**	**Good for classification**	**High computation**
Random Forest	**98%**	**97%**	**97%**	**97%**	**High accuracy, robust**	**Slightly slower**
LSTM	**98.4%**	**97.8%**	**98%**	**97.9%**	**Handles sequence patterns**	**High resource usage**
Ensemble Model	**98.6%**	**98%**	**98%**	**98%**	**Best overall performance**	**Complex implementation**

Description:
Table 5.1 compares different machine learning models used in the project. The ensemble model shows the best performance by combining multiple models, achieving high accuracy and better generalization.

5.4 ANALYSIS AND INTERPRETATION OF RESULTS

The experimental results obtained from the implementation provide strong evidence that the proposed system is effective in identifying malicious URLs with high accuracy and reliability. The performance of different models indicates that each algorithm contributes uniquely to the detection process. Traditional machine learning models such as Random Forest and SVM perform well in capturing statistical patterns and structured features, while the LSTM model adds value by identifying sequential patterns within URL strings.

One of the key observations from the results is that lexical features alone are capable of achieving a relatively high level of accuracy. This suggests that many malicious URLs exhibit recognizable structural characteristics such as unusual length, excessive use of special characters, or suspicious tokens. However, when additional features such as host-based and content-based attributes are incorporated, the model performance improves further. This confirms that a multi-dimensional feature approach is more effective than relying on a single feature category.

The LSTM model shows strong performance in detecting complex and obfuscated URLs, which are often difficult to identify using conventional methods. Its ability to understand sequence patterns makes it particularly useful in handling evolving attack strategies. However, it is also observed that LSTM requires more computational resources and training time, which can affect scalability.

The ensemble model provides the most balanced and optimal results. By combining the outputs of different models, it reduces individual model biases and improves overall prediction accuracy. The low false positive rate indicates that the system does not frequently misclassify legitimate URLs, which is important for maintaining user trust. At the same time, the high recall value ensures that most malicious URLs are successfully detected, reducing the risk of security breaches.

5.5 DISCUSSION ON OUTCOMES

The outcomes of this project highlight the practical advantages of using machine learning techniques for cybersecurity applications, particularly in detecting malicious URLs. Compared to traditional blacklist-based methods, which rely on previously known threats, the proposed

system demonstrates the ability to detect new and unseen malicious URLs. This capability is especially important in addressing zero-day attacks, where threats emerge rapidly and cannot be captured by static databases.

One of the major strengths of the system is its adaptability. Since the models are trained on diverse datasets with multiple feature categories, the system is capable of generalizing well to different types of malicious URLs. The use of ensemble learning further enhances this adaptability by combining different perspectives of analysis into a single decision-making process.

At the same time, the project also reveals certain challenges. For example, incorporating content-based features improves detection accuracy but increases computational overhead. This may affect performance in large-scale or real-time deployments where speed is critical. Similarly, deep learning models like LSTM require more memory and processing power, which can limit their usage in resource-constrained environments.

Overall, the outcomes demonstrate that the system is robust, efficient, and capable of addressing modern cybersecurity challenges. It provides a scalable solution that can be further enhanced with additional features, larger datasets, and advanced models.

5.6 SUMMARY

This chapter presented a comprehensive evaluation of the proposed malicious URL detection system. It began with a detailed description of the experimental setup and environment, followed by an explanation of the performance metrics used for evaluation. A comparative analysis of different models highlighted the strengths and limitations of each approach, with the ensemble model achieving the best overall performance.

The analysis and interpretation of results provided insights into how different features and algorithms contribute to the system's effectiveness. The discussion on outcomes emphasized the practical relevance of the system in real-world cybersecurity scenarios, while also identifying areas for improvement.

In conclusion, the results confirm that the proposed approach successfully meets the objectives of the project by providing accurate, fast, and reliable detection of malicious URLs. The

findings of this chapter serve as a strong foundation for the final conclusions and future work discussed in the next chapter.

CHAPTER 6 CONCLUSION AND FUTURE WORK

6.1 SUMMARY OF CONTRIBUTIONS

This project makes a significant contribution to the field of cybersecurity by proposing an effective and practical solution for the early detection of malicious URLs using machine learning techniques. One of the primary contributions is the development of a structured and end-to-end framework that systematically handles the entire process—from data collection and preprocessing to model training, evaluation, and real-time prediction. This complete pipeline ensures that the system is not only theoretically sound but also practically implementable.

A major contribution of this work is the use of a diverse feature set for URL analysis. The system incorporates lexical features derived directly from the URL structure, host-based features obtained from domain-related information, and content-based features extracted from webpage elements. By combining these different types of features, the system is able to capture multiple dimensions of malicious behavior, leading to improved detection accuracy and robustness.

Another important contribution is the implementation and comparison of multiple machine learning and deep learning models, including Random Forest, Support Vector Machine, and Long Short-Term Memory networks. Each of these models offers unique advantages, and their combined use provides a deeper understanding of how different approaches perform in the context of malicious URL detection. The introduction of an ensemble model that integrates the predictions of these algorithms further enhances the system's performance by reducing individual model weaknesses and improving overall reliability.

The project also contributes by demonstrating the importance of feature engineering and data preprocessing in building effective machine learning systems. The transformation of raw URL data into structured numerical representations plays a crucial role in enabling models to learn meaningful patterns. This highlights the practical aspect of machine learning beyond just model selection.

In addition, the system is designed with real-time applicability in mind. The low latency achieved during prediction ensures that the model can be used in real-world scenarios such as browser-based security tools or network monitoring systems. This makes the solution not only accurate but also efficient and user-friendly.

Finally, the project provides a scalable foundation for future research and development. The modular design allows for easy integration of new features, datasets, or advanced models, making it adaptable to evolving cybersecurity threats. Overall, the contributions of this project lie in its comprehensive approach, practical implementation, and its ability to address real-world challenges in malicious URL detection.

6.2 KEY FINDINGS

The study conducted in this project led to several important findings that highlight the effectiveness of machine learning approaches in detecting malicious URLs. One of the most significant findings is that properly engineered features play a crucial role in achieving high detection accuracy. It was observed that even simple lexical features, such as URL length, number of special characters, and presence of suspicious keywords, can provide strong initial indicators of malicious intent. These features alone were capable of delivering reasonably high accuracy, demonstrating their importance in early-stage detection.

Another key finding is that combining multiple types of features significantly improves model performance. While lexical features capture structural patterns, host-based features provide contextual information such as domain age and DNS characteristics, and contentbased features offer deeper insights into webpage behavior. The integration of these feature categories enables the system to detect a wider range of malicious URLs, including those that attempt to evade detection through obfuscation or disguise.

The evaluation of different models revealed that no single algorithm is universally superior in all aspects. Traditional machine learning models like Random Forest and Support Vector Machine showed strong performance in terms of stability, speed, and accuracy. On the other hand, the LSTM model demonstrated its strength in capturing sequential dependencies within URL strings, making it particularly effective in identifying complex and obfuscated URLs. This highlights the importance of selecting models based on the nature of the problem rather than relying on a single technique.

A major finding of this project is the effectiveness of the ensemble approach. By combining the predictions of multiple models, the system achieves higher accuracy and better generalization compared to individual models. The ensemble model reduces the chances of misclassification by balancing the strengths and weaknesses of different algorithms. This approach proved to be the most reliable, achieving the best overall performance across all evaluation metrics.

Another important observation is related to the system's real-time capability. The model was able to classify URLs within a very short time, maintaining low latency while still delivering high accuracy. This confirms that machine learning-based solutions can be effectively deployed in real-world applications such as browser extensions or network security systems without causing noticeable delays.

Finally, the study highlights that malicious URL detection is an evolving challenge. Attackers continuously modify their techniques to bypass detection systems, which means that models must be regularly updated and retrained with new data. This finding emphasizes the need for adaptive and scalable solutions that can keep pace with changing threat landscapes.

Overall, the key findings of this project demonstrate that a combination of well-engineered features, multiple learning models, and ensemble techniques can significantly enhance the detection of malicious URLs, making the system both effective and practical for real-world cybersecurity applications.

6.3 LIMITATIONS OF THE STUDY

While the proposed system for early detection of malicious URLs using machine learning demonstrates strong performance, it is important to acknowledge certain limitations that affect its scope, generalization, and real-world applicability. Identifying these limitations not only ensures transparency but also helps in guiding future improvements.

One of the primary limitations of this study lies in the **dependency on dataset quality and diversity**. The model has been trained on a dataset of labelled URLs, which, although sufficiently large, may not fully represent the constantly evolving nature of cyber threats. Attackers frequently generate new URL patterns using advanced techniques such as domain generation algorithms (DGAs), URL obfuscation, and polymorphic structures. As a result, the model may not always generalize perfectly to unseen or highly novel attack patterns, especially zero-day threats that differ significantly from training data.

Another limitation is related to the **feature extraction process**, particularly for host-based and content-based features. Some of these features rely on external sources such as WHOIS data,

DNS records, or webpage content retrieval. In real-time environments, accessing such information can introduce **latency, dependency on network availability, and occasional missing data**. This may affect the speed and reliability of classification, especially when the system is deployed in high-throughput environments like browsers or network gateways.

The use of **deep learning components (such as LSTM)** also introduces computational constraints. Although the hybrid model improves detection accuracy, it increases **processing complexity and resource consumption** compared to simpler models like Logistic Regression or Decision Trees. This creates challenges for deployment in low-resource environments such as mobile devices or edge systems, where memory and processing power are limited.

Another important limitation is the **risk of adversarial manipulation**. Machine learning models can be intentionally deceived by attackers who craft URLs specifically to bypass detection systems. Techniques such as adding benign-looking tokens, increasing URL length, or mimicking legitimate domain structures can reduce the confidence of classifiers. While the system performs well under normal conditions, it may still be vulnerable to carefully engineered adversarial inputs.

The study also has limitations in terms of **lack of real-time continuous learning**. The current model is trained offline and does not automatically update itself when new types of malicious URLs emerge. Without periodic retraining, the system may gradually become less effective over time due to concept drift in attack patterns.

Additionally, the **evaluation environment is controlled**, meaning the results are based on experimental datasets rather than fully dynamic, real-world internet traffic. Factors such as network noise, encrypted traffic, user behavior variations, and large-scale deployment constraints are not fully captured in the current evaluation.

Lastly, while the system focuses on URL-based detection, it does not deeply analyze **user behavior, device-level signals, or contextual cybersecurity intelligence**, which could further enhance detection accuracy. This makes the system effective as a standalone detection layer but not a complete cybersecurity solution.

In summary, although the proposed approach achieves high accuracy and demonstrates strong potential for real-time malicious URL detection, its limitations highlight the need for continuous dataset updates, improved robustness against adversarial attacks, optimized computational efficiency, and integration with broader security frameworks. These aspects form the foundation for future enhancements of the system.

6.4 SUGGESTIONS FOR FUTURE RESEARCH

While the current project demonstrates an effective approach for detecting malicious URLs using machine learning, there are several areas where the system can be further improved and extended. These directions not only enhance performance but also make the system more robust, scalable, and suitable for real-world deployment.

One important direction for future research is the **integration of advanced deep learning architectures**, particularly transformer-based models such as BERT-like frameworks adapted for URL understanding. These models have shown strong capability in capturing contextual relationships within sequences and could improve detection accuracy, especially for complex and obfuscated URLs. However, efforts should also focus on optimizing such models through techniques like model compression or knowledge distillation to ensure they remain deployable in real-time environments.

Another key improvement area is **real-time adaptive learning**. Instead of relying solely on offline training, the system can be enhanced with online learning mechanisms that allow it to continuously update its knowledge based on newly observed URL patterns. This would help the model stay relevant against evolving cyber threats and reduce the impact of concept drift over time.

Future work can also explore **adversarially robust machine learning techniques**. Since attackers actively attempt to evade detection systems, incorporating adversarial training—where the model is exposed to intentionally modified malicious URLs—can significantly improve resilience. Techniques such as Generative Adversarial Networks (GANs) can be used to generate realistic adversarial samples for training more secure models.

Another promising direction is the **use of multi-modal feature integration**. Currently, the system focuses mainly on lexical, host-based, and limited content features. Future systems could incorporate additional signals such as user behavior patterns, clickstream data, browser interaction data, and network traffic characteristics. Combining these diverse data sources would provide a more comprehensive understanding of malicious activity and improve detection performance.

Scalability is also an important aspect for future research. The system can be extended to support **large-scale deployment in cloud or distributed environments**, enabling it to handle millions of URL requests in real time. Technologies such as distributed computing frameworks and edge-based processing can be explored to balance performance and latency.

Another area of improvement is the **development of lightweight models for edge and mobile devices**. As cybersecurity solutions are increasingly required on personal devices, optimizing the model to run efficiently with minimal computational resources will make the system more accessible and widely usable.

In addition, future research can focus on **explainable AI (XAI)** techniques. Providing clear explanations for why a URL is classified as malicious or benign can improve user trust and help cybersecurity professionals better understand attack patterns. This is especially important in sensitive applications where transparency and accountability are required.

The system can also be expanded into a **complete cybersecurity framework**, integrating with browser extensions, firewalls, intrusion detection systems, and enterprise security tools. This would transform the project from a standalone classifier into a practical, real-world security solution.

Finally, continuous benchmarking with updated datasets and collaboration with cybersecurity organizations can ensure that the system remains aligned with current threat landscapes. Incorporating threat intelligence feeds and real-world attack data will further strengthen the model's effectiveness.

In summary, future research should focus on improving adaptability, robustness, scalability, interpretability, and real-world integration. By addressing these areas, the proposed system can evolve into a more comprehensive and intelligent solution for combating malicious URLbased cyber threats.

6.5 SUMMARY

This chapter presented the overall conclusion and future scope of the project on early detection of malicious URLs using machine learning techniques. It began by summarizing the key contributions of the work, highlighting how the proposed system effectively combines multiple

feature types and machine learning models to achieve high detection accuracy. The important findings of the study were then discussed, showing that the system performs efficiently in identifying malicious URLs with low latency, making it suitable for real-time applications.

The chapter also examined the limitations of the current approach, such as dependency on dataset quality, computational complexity of advanced models, and challenges in handling newly emerging attack patterns. Recognizing these limitations helps in understanding where improvements are required and ensures the transparency of the research work.

Finally, suggestions for future research were outlined, focusing on areas like advanced deep learning models, real-time adaptive systems, adversarial robustness, scalability, and integration with broader cybersecurity frameworks. These improvements can significantly enhance the effectiveness and practicality of the system.

Overall, this chapter concludes that while the proposed system successfully meets its objectives and demonstrates strong performance, there is still room for enhancement to make it more robust, scalable, and adaptable to the continuously evolving landscape of cyber threats

REFERENCES

[1] M. Aljabri et al., "Detecting malicious URLs using machine learning techniques: review and research directions," IEEE Access, vol. 10, pp. 121395–121417, 2022. DOI: 10.1109/ACCESS.2022.3222307

[2] A. Weshahi, F. Dwaik, M. Khouli, H.I. Ashqar, and A. Shatnawi, "IoT-enhanced malicious URL detection using machine learning," in Proc. International Conference on Advanced Computing and Applications (ACANAC), Springer, 2024.

[3] M. Swarnkar, N. Sharma, and H.K. Thakkar, "Malicious URL detection using machine learning," Studies in Computational Intelligence, vol. 1065, pp. 199–216, 2023. DOI: 10.1007/978-981-19-6290-5_11

[4] S. Das Guptta, K.T. Shahriar, H. Alqahtani, D. Alsalman, and I.H. Sarker, "Modeling hybrid feature-based phishing websites detection using machine learning techniques," Annals of Data Science, vol. 11, no. 1, pp. 217–242, 2024. DOI: 10.1007/S40745-02200379-8

[5] A. Karim, M. Shahroz, K. Mustofa, S.B. Belhaouari, and S.R.K. Joga, "Phishing detection system through hybrid machine learning based on URL," IEEE Access, vol. 11, pp. 36805–36822, 2023.

[6] R. Alazaidah et al., "Website phishing detection using machine learning techniques," Journal of Statistics and Applications in Probability, vol. 13, no. 1, pp. 119–129, 2024.

[7] M. Alsaedi, F.A. Ghaleb, F. Saeed, J. Ahmad, and M. Alasli, "Cyber threat intelligencebased malicious URL detection model using ensemble learning," Sensors, vol. 22, p. 3373, 2022.

[8] S.S. Roy, A.I. Awad, L.A. Amare, M.T. Erkihun, and M. Anas, "Multimodel phishing URL detection using LSTM, bidirectional LSTM, and GRU models," Future Internet, vol. 14, no. 11, p. 340, 2022. DOI: 10.3390/FI14110340

[9] Z. Alshingiti, R. Alaqel, J. Al-Muhtadi, Q.E.U. Haq, K. Saleem, and M.H. Faheem, "A deep learning-based phishing detection system using CNN, LSTM, and LSTM-CNN," Electronics, vol. 12, no. 1, p. 232, 2023. DOI: 10.3390/ELECTRONICS12010232

[10] R. Liu, Y. Wang, Z. Guo, H. Xu, Z. Qin, W. Ma, and F. Zhang, "TransURL: Improving malicious URL detection with multi-layer Transformer encoding and multi-scale pyramid features," Computer Networks, vol. 253, p. 110707, 2024.

[11] M.-Y. Su and K.-L. Su, "BERT-based approaches to identifying malicious URLs," Sensors, vol. 23, p. 8499, 2023.

[12] M. Elsadig, A.O. Ibrahim, S. Basheer, M.A. Alohali, S. Alshunaifi, H. Alqahtani, N. Alharbi, and W. Nagmeldin, "Intelligent deep machine learning cyber phishing URL detection based on BERT features extraction," Electronics, vol. 11, p. 3647, 2022.

[13] H. Kibriya, A. Siddiqa, W.Z. Khan, and M.K. Khan, "Towards safer online communities: deep learning and explainable AI for hate speech detection and classification," Computers & Electrical Engineering, vol. 116, p. 109153, 2024.

[14] T.L. Halili, A. Halili, K. Vishi, and B. Rexha, "NoPhish: Efficient Chrome extension for phishing detection using machine learning techniques," arXiv: Cryptography and Security, 2024.

[15] M. Sanchez-Paniagua, E.F. Fernandez, E. Alegre, W. Al-Nabki, and V. Gonzalez-Castro, "Phishing URL detection: A real-case scenario through login URLs," IEEE Access, vol. 10, 2022.

[16] S.D. Gupta, K.T. Shahriar, and I.H. Sarker, "Phishing detection using CNN with Brown Bear optimization," in Proc. 2023 International Conference, 2024.

[17] S.R. Abdul Samad, A. Abdullah, M.F. Abdollah, A.A. Mutalib, and S.A.M. Noah, "Analysis of the performance impact of fine-tuned machine learning model for phishing URL detection," Electronics, vol. 12, no. 7, 2023.

[18] A.M. Salih et al., "A perspective on explainable artificial intelligence methods: SHAP and LIME," Advanced Intelligent Systems, vol. 7, p. 2400304, 2025.

[19] A. Saxena, A. Arora, S. Saxena, and A. Kumar, "Analysis of web security flaws and machine learning approaches forURL classification," in Proc. 2022 International Conference on Intelligent Computing and Communication, 2022.

[20] C. Catal, G. Giray, B. Tekinerdogan, S. Kumar, and S. Shukla, "Applications of deep learning for phishing detection: a systematic literature review," Artificial Intelligence Review, vol. 64, no. 6, 2022.

[21] Y. Gu et al., "From past to present: A survey of malicious URL detection techniques, datasets and code repositories," arXiv:2504.16449, 2025.

[22] S. Aslam, H. Aslam, A. Manzoor, H. Chen, and A. Rasool, "AntiPhishStack: LSTMbased stacked generalization model for optimized phishing URL detection," Symmetry, vol. 16, no. 2, 2024.

[23] I. Kara, M. Ok, and A. Ozaday, "Characteristics of understanding URLs and domain names features: The detection of phishing websites with machine learning methods," IEEE Access, 2022.

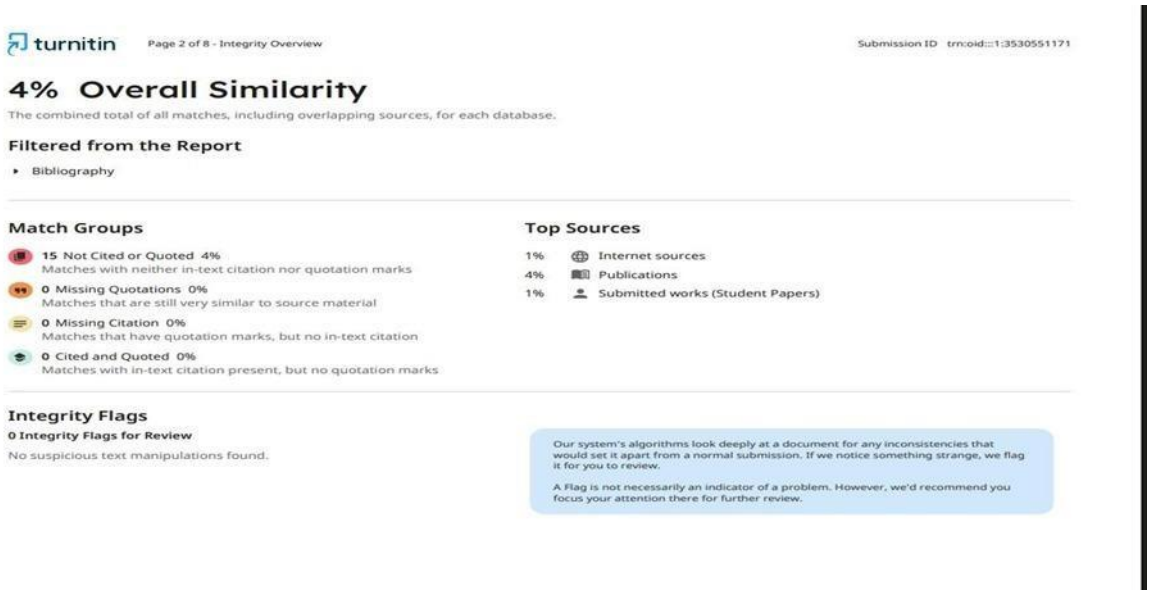
[24] Anti-Phishing Working Group (APWG), "Phishing Activity Trends Report, Q4 2023," APWG, 2024. [Online]. Available: https://apwg.org/trendsreports

BASE PAPER

[1] Y. Gu et al., "From past to present: A survey of malicious URL detection techniques, datasets and code repositories," arXiv:2504.16449, 2025.

APPENDIX A

A.1 Project Similarity Report


turnitin Page 2 of 8 - Integrity Overview Submission ID trn:oid:::1:3530551171

4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- 15 Not Cited or Quoted 4%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
4% Publications
1% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Fig A.1 similarity Report

The similarity report provides an overview of the originality of the project document by indicating the percentage of content that matches existing sources. It highlights the extent of textual overlap and helps in identifying whether proper citation and referencing have been followed. A lower similarity index reflects that the work is largely original, while any matched content is either commonly used technical terminology or appropriately referenced material. This report ensures that the project maintains academic integrity and adheres to plagiarism guidelines.

APPENDIX B

B.1 Dataset and Access Control

The dataset used in this project consists of a large collection of labeled URLs, including both safe (benign) and malicious links. These URLs are gathered from publicly available sources such as trusted websites and phishing detection repositories. The dataset is organized in a structured format, where each URL is assigned a label indicating whether it is safe or harmful, enabling the system to learn patterns effectively.

The dataset is processed by extracting important features such as URL structure, domain details, and behavioral characteristics. These features are then cleaned, normalized, and prepared for

model training. The processed data is used to train and evaluate the machine learning models, and it can be easily updated by adding new URL data to improve the system's performance over time.

APPENDIX C

C.1 Research paper

Early Detection of Malicious URLs Using Machine Learning Algorithms

Bhagyalakxmi
Dept. of Information Science and Technology,
Presidency University, Bengaluru, India
BHAGYALAKXMI.20221IST0054@presidencyuniversity.in

Yashaswini D
Dept. of Information Science and Technology,
Presidency University, Bengaluru, India
YASHASWINI.20221ist0083@presidencyuniversity.in

Dr. Debasmita Mishra
Dept. of Computer Science and Engineering,
Presidency University, Bengaluru, India
debasmita@presidencyuniversity.in

Sanjana C Rao
Dept. of Information Science and Technology,
Presidency University, Bengaluru, India
SANJANA.20221IST0072@presidencyuniversity.in

Abstract—Internet spread has resulted in a rapid number of cyber threats, and malicious URLs have become the most common vehicle in the spread of phishing, malware, and other cybersecurity attacks. The detection systems that are based on blacklisting and signature detection are becoming ineffective in detecting dynamically generated and obfuscated malicious links. This paper introduces a detailed machine learning based system to detect malicious URLs early in the lexical, structural and entropy-based features directly obtained in URL strings, that is, no traffic to the pages is needed. We design 35 URL features and test six classical machine learning models, which are Support Vector Machine (SVM), Naive Bayes, Decision Tree, Random Forest, Logistic Regression and XGBoost. Our XGBoost model has the highest current accuracy of 99.99% with an AUC-ROC of 0.9992 and the correct classification of 61,274 benign and 15,285 malicious URLs in the test set. To offer model

leads users to the malicious destinations - sites designed to steal credentials, distribute malware or phish, launch ransomware attacks or steal sensitive information.

The Anti-Phishing Working Group (APWG) reports that phishing activities are increasingly becoming alarming, with more than 4.7 million unique phishing websites observed in the year 2023 alone, an increase of 150 percent over the years since 2019. Cybercrime costs organizations and individuals hundreds of billions of dollars annually all over the world, and URL-based attacks represent a considerable part of the incidences. Even security conscious users are fooled by social engineering attacks, especially those that involve misleading URL names that appear to represent trusted services like banking websites, online stores and government websites.

Conventional defense systems against bad URLs are

Fig C.1 Research paper

APPENDIX D

D.1 Github Repository

https://github.com/Bhagyaaaaa/Early-Detection-of-Malicious-URLs-Using-MachineLearning-Algorithms

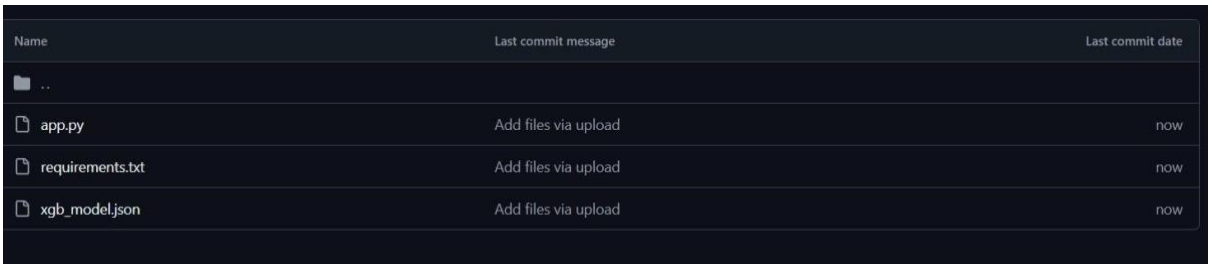


Fig D.1 Research paper